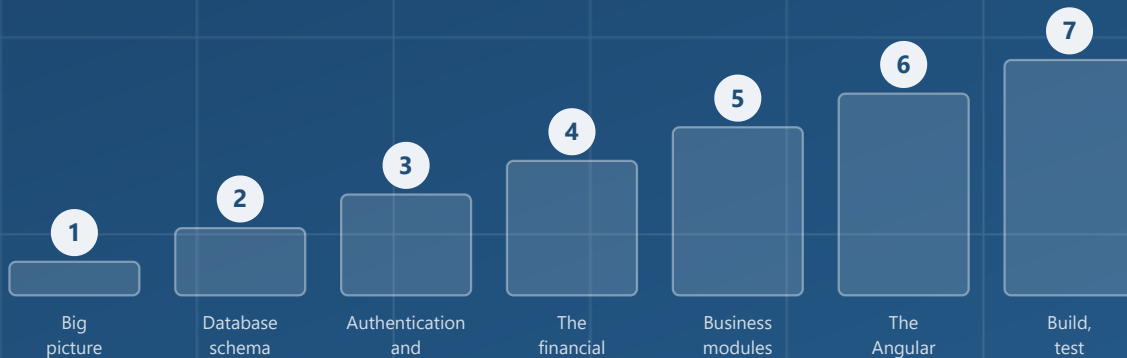


explain dabadaba

Your own project, explained from zero. One step after another, with the real code and a picture for every part.



daba daba — Tunisian for *step by step, one step after another*. That is how this guide is built. No page expects you to already know what was on the page before.

How to read this guide

You built this project. This guide is here so that you can also *explain* it — to a jury, to a workmate, or to yourself in six months when you have forgotten how a part of it works.

Every chapter has the same seven parts. So once you have read one chapter, you already know how to read the other six:

1 In one sentence

What this part of the system does, said with no technical words at all.

2 Think of it like...

A comparison with something from real life. Once the comparison makes sense, the code stops being scary.

3 Why it exists

What would break, or stay stuck in Excel, if this part was not there.

4 The picture

One drawing. Look at it before you read the steps. It is your map.

5 Step by step

What happens, cut into numbered steps, with the real file names.

6 The real code

Short pieces of your own code. After each one there is a plain explanation. Nothing here is made up: every line was copied from your project.

7 Words & traps

A small dictionary of the words used, then the mistakes a beginner makes when reading this code.

ONE THING TO KNOW FIRST

The code mixes English and French on purpose. The technical names are English (ProjectService, findAll). The business words stay French, because French is what people at ST2i say out loud: **jalon** = a billing step, **charge** = work measured in days, **avenant** = a change to the contract, **devis interne** = the internal cost estimate, **TCC** = how much one person costs per year. Keeping these words in French means the screen says the same thing as the people in the office.

The project in numbers

192 JAVA FILES	10 771 LINES OF JAVA	66 TYPESCRIPT FILES	11 389 LINES OF TS
28 DB MIGRATIONS	14 TEST CLASSES	26 PERMISSION CODES	4 ROLES

The seven chapters

- 1 Big picture and overall architecture (three tiers: Angular frontend, Spring Boot backend, PostgreSQL database)**

PMS is one web application built in three separate pieces — the screen you click on, the server that decides and calculates, and the database that remembers — so the margin calculations ST2i used to do in an Excel workbook now happen in one shared place, with rules and with permissions.
 - 2 Database schema and Java-to-table mapping (PostgreSQL + Flyway migrations + JPA/Hibernate entities)**

This is the part of PMS that keeps everything for good: it says what the filing cabinets (the tables) inside PostgreSQL look like, it keeps their shape under version control so every copy of the application has the same shape, and it turns Java objects into rows of those tables and back.
 - 3 Authentication and Authorization (login, JWT, permissions, project scope)**

This part of PMS checks that you really are the person you say you are when you log in, and then, on every single request, it decides which screens you may open and whose project figures you may see.
 - 4 The financial engine (KPI calculation, milestone billing and budget revisions)**

It takes the days people work, and what each of those days costs the company, and turns them into live money figures: what the project has already spent, what it will cost in total by the end, and how much of the price sold to the client is left over as profit.
 - 5 Business modules and the rules they enforce (project, workload, billing, governance, mission, agile)**

These modules store everything you need to follow a consulting project — its identity card and budget, who was planned to work on it, who really worked on it, when the client can be invoiced, what risks and deliverables exist, who travelled, and what the team still has to build. And they refuse any change that would break a company rule, for example invoicing more than 100 percent of the contract.
-

6 The Angular frontend (pms-frontend): routing, shell layout, signals, services, HTTP interceptor, and the project-picker pattern

It is the part of PMS that runs inside the user's web browser: it draws every screen, it chooses which screen the address bar should show, it adds the login token to every call it sends to the server, and it turns the answers back into tables, cards and forms.

7 Build, test and delivery pipeline (Docker, nginx, GitHub Actions, Maven/JaCoCo)

It packs the whole application — database, API and web interface — into three sealed boxes that start with one single command, and it runs an automatic checklist on every code change, so a broken version never reaches a server.

Big picture and overall architecture (three tiers: Angular frontend, Spring Boot backend, PostgreSQL database)

PMS is one web application built in three separate pieces — the screen you click on, the server that decides and calculates, and the database that remembers — so the margin calculations ST2i used to do in an Excel workbook now happen in one shared place, with rules and with permissions.

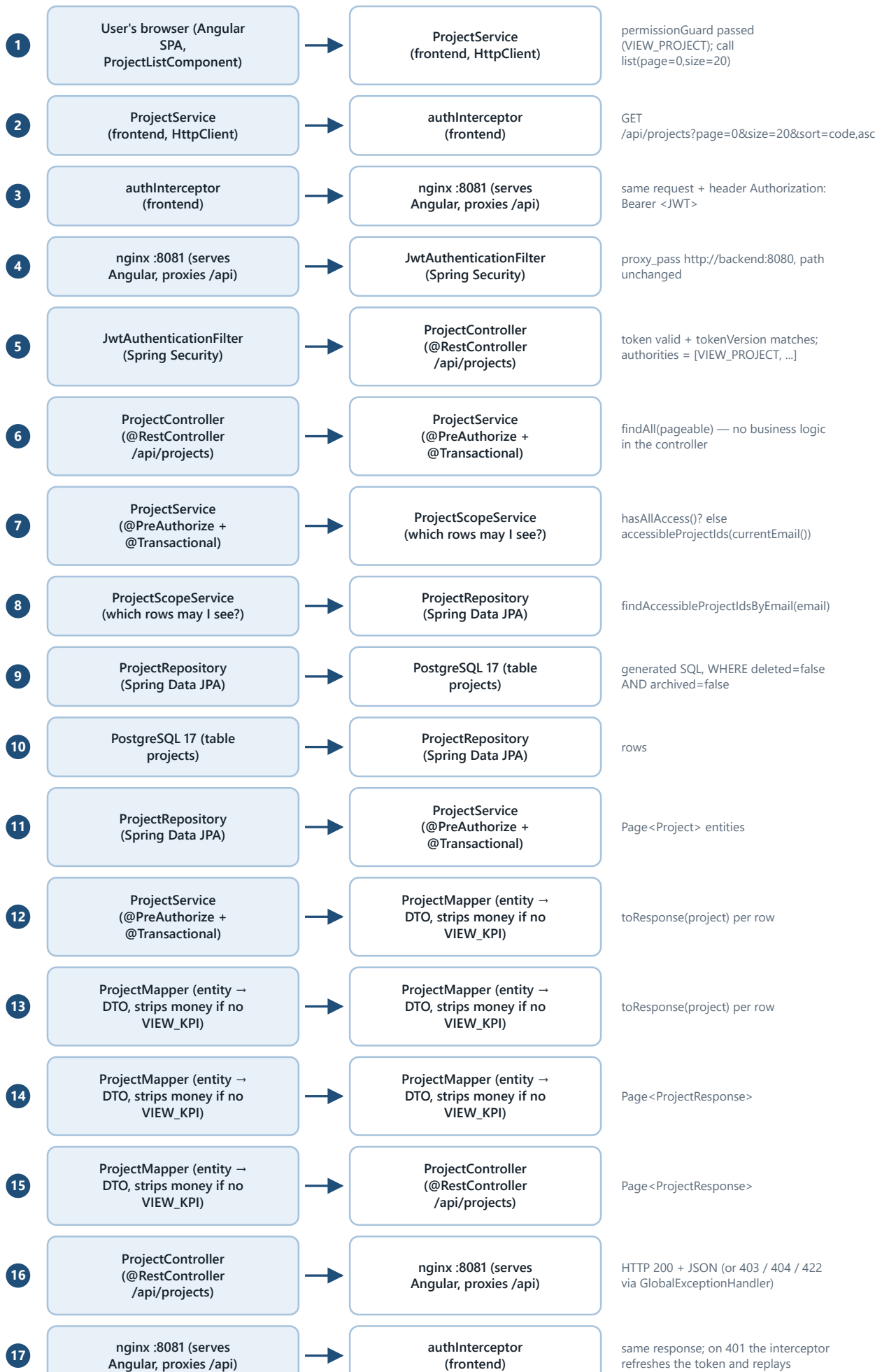
THINK OF IT LIKE...

Think of a restaurant. The dining room is the Angular frontend. It is the only part the customer sees. It shows the menu and takes the order, but it cooks nothing. The kitchen is the Spring Boot backend. It reads the order, checks if the dish is allowed today, does the real work, and says no to an order it cannot make. The pantry is the PostgreSQL database. It only keeps the ingredients on labelled shelves and hands them over when the kitchen asks. A waiter (the nginx reverse proxy) carries the paper slips between the dining room and the kitchen, and the customer never walks into the kitchen himself.

WHY THIS EXISTS

ST2i managed project margins in an Excel workbook (`DEV_Fiche_revue_projet-Real.xlsx`). One workbook means only one person can edit at a time. It also means no access control, formulas that break (the real file still has `#REF!` and `#VALUE!` cells), and no history of changes. Splitting the same job into three tiers fixes exactly those problems. The database gives one shared copy of the numbers that is always up to date. The backend keeps the formulas and the business rules in one place, so nobody can "edit the formula in their own copy". And the frontend can show a director the full margin while hiding it from a developer, which a spreadsheet cell can never do. `DECISIONS.md` ADR-002 writes down that the team chose to build the *full* Excel financial model again (EV, ETC, EAC, three margins), not a smaller version. So the platform is a real replacement and not a step down.

The picture



18

authInterceptor
(frontend)User's browser (Angular
SPA,
ProjectListComponent)

Observable emits; the table renders

One click, one round trip: browser → nginx → Spring Boot → PostgreSQL → back — Look at the arrows: they only go one way down and one way back. The browser never reaches PostgreSQL. The controller never reaches the repository. And the choice to hide the budget is made in the service layer — so the numbers a developer is not allowed to see never even leave the server.

Step by step

1

You click "Projects" in the browser

The Angular application is already loaded in your browser. Angular is a framework for building a SPA — a Single Page Application. That means one HTML page that rewrites parts of itself instead of asking the server for a new page on every click. Your click matches a route (a rule that links a URL to a screen) in `app.routes.ts`. Before the screen is even loaded, a guard (a small function that can stop the move) named `permissionGuard` checks that your account has the capability `VIEW_PROJECT`. The route entry really does carry `data: { permission: 'VIEW_PROJECT' }`.

FILES `frontend/pms-frontend/src/app/app.routes.ts` · `frontend/pms-frontend/src/app/core/guards/permission.guard.ts`

2

The screen asks a service for the data, and the service builds an HTTP call

The screen component never talks to the network itself. It calls `ProjectService.list(...)`, an Angular service — a reusable object that Angular builds once and hands to whoever needs it. That service turns the request into `GET /api/projects?page=0&size=20&sort=code,asc`. Note that it only builds the address and the query parameters. No rule, no calculation and no margin is worked out here.

FILES `frontend/pms-frontend/src/app/core/services/project.service.ts`

3

An interceptor staples your identity card onto the request

Every call going out passes through `authInterceptor`, an HTTP interceptor — a function that sees each request on its way out and each answer on its way back. It adds the header `Authorization: Bearer <token>`. The token is a JWT (JSON Web Token: a signed piece of text that proves who you are without the server keeping a session for you). If the server later answers 401 ("your token has expired"), the same interceptor quietly asks for a fresh token and sends the first request again. So the user is never logged out for no clear reason.

FILES `frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts` · `frontend/pms-frontend/src/app/core/services/auth.service.ts`

4

nginx gets the call and passes anything starting with /api to the backend

In the Docker setup, the browser only ever talks to one address, `http://localhost:8081`. That address is nginx, a web server working as a reverse proxy — a middleman that answers some requests itself and passes others to another server. Requests for the Angular files it answers from disk. Requests that start with `/api/` it passes on to `http://backend:8080`. Because the app and the API share the same address, the browser sees a single origin. So no CORS rules are needed, and the refresh cookie marked `SameSite=Strict` just works.

FILES `frontend/pms-frontend/docker/nginx.conf` · `docker-compose.yml`

5

Spring Security reads the token before any of your code runs

Inside the backend, `JwtAuthenticationFilter` runs first. A filter is code that looks at every request before it reaches the controller. It reads the token, checks the signature, and loads the user's permissions from the database (kept in memory for five minutes). It also compares the token's `tokenVersion` with the user's current version, so an administrator can make somebody's tokens useless right away. What comes out is a list of authorities — plain pieces of text such as `VIEW_PROJECT` — attached to the request.

FILES `backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java` · `backend/src/main/java/com/pms/shared/config/SecurityConfig.java`

6

The controller gets the request and passes it on at once

`ProjectController` is marked `@RestController` and `@RequestMapping("/api/projects")`, which means "this class answers HTTP calls whose path starts with `/api/projects`, and it returns JSON". Its job is small on purpose: read the URL and the body, hand them to the service, and wrap what comes back in an HTTP status code. There is not one single business rule in this file — and that is the whole point of the first layer.

FILES `backend/src/main/java/com/pms/project/controller/ProjectController.java`

7

The service decides: first the permission, then which rows, then the rule

`ProjectService` is the layer that thinks. `@PreAuthorize("hasAuthority('VIEW_PROJECT')")` refuses the call at once if the capability is missing. Then `ProjectScopeService` cuts down *which rows* you may see: a director sees everything, a project manager sees only the projects he leads, and a developer sees only the ones he is assigned to. `@Transactional` means everything inside the method works together or is undone together. Other methods hold real business rules. For example, `archive()` throws "Seul un projet terminé peut être archivé" if the project is not `COMPLETED`, and `changeStatus()` says no to any change the `ProjectStatus` enum does not list as allowed.

FILES `backend/src/main/java/com/pms/project/service/ProjectService.java` · `backend/src/main/java/com/pms/project/service/ProjectScopeService.java` · `backend/src/main/java/com/pms/project/entity/ProjectStatus.java`

The repository turns a method name into SQL, and the answer comes back as DTOs

`ProjectRepository` is only an interface — you say what you want, and Spring Data JPA writes the SQL for you. JPA is the Java standard for linking objects to table rows, so a `Project` object and a row of the `projects` table are two views of the same thing. The rows come back as `Project` entities. Then `ProjectMapper` (made by MapStruct when the code is compiled) copies them into `ProjectResponse` DTOs. A DTO is a plain object that only carries data, shaped for the outside world. Entities never leave the service layer. On the way out, `toResponse()` takes out every money field when the caller does not have `VIEW_KPI`, so a developer really does get JSON with no budget in it. Angular then draws the table.

```
FILES backend/src/main/java/com/pms/project/repository/ProjectRepository.java ·
      backend/src/main/java/com/pms/project/mapper/ProjectMapper.java ·
      backend/src/main/java/com/pms/project/dto/ProjectResponse.java
```

The real code

backend/ and frontend/ (folder layout, as listed on disk)

TEXT

```
1 PMS/
2 |─ docker-compose.yml          3 containers: db + backend + frontend
3 |─ backend/src/main/
4 |   |─ java/com/pms/
5 |   |   |─ PmsApplication.java    the single entry point
6 |   |   |─ agile/      controller/ dto/ entity/ mapper/ repository/ service/
7 |   |   |─ auth/       controller/ dto/ security/ service/
8 |   |   |─ billing/    controller/ dto/ entity/ mapper/ repository/ service/
9 |   |   |─ governance/ controller/ dto/ entity/ mapper/ repository/ service/
10 |   |   |─ kpi/        controller/ dto/ entity/ mapper/ repository/ service/
11 |   |   |─ mission/    controller/ dto/ entity/ mapper/ repository/ service/
12 |   |   |─ project/    controller/ dto/ entity/ mapper/ repository/ service/
13 |   |   |─ team/       controller/ dto/ entity/ mapper/ repository/ service/
14 |   |   |─ user/       controller/ dto/ entity/ mapper/ repository/ service/
15 |   |   |─ workload/   controller/ dto/ entity/ mapper/ repository/ service/
16 |   |   |─ shared/     config/ entity/ exception/ response/ security/
17 |   |─ resources/
18 |   |   |─ application.yml  application-dev.yml  application-prod.yml
19 |   |   |─ db/migration/    V1__schema_auth.sql ... V28__backlog_item_assignee.sql
20 |─ frontend/pms-frontend/
21 |   |─ docker/nginx.conf    serves Angular + proxies /api
22 |   |─ src/app/
23 |   |   |─ app.config.ts  app.routes.ts  app.ts
24 |   |   |─ core/         guards/ i18n/ interceptors/ models/ services/
25 |   |   |─ features/     admin/ agile/ auth/ billing/ dashboard/ di/
26 |   |   |               governance/ kpi/ missions/ projects/ resources/ workload/
27 |   |   |─ layout/       shell/ sidebar/ language-switcher/
28 |   |   |─ shared/       confirm-modal/ pagination/ project-picker/ toast-container/
```

What this does. Read the backend list from top to bottom. Every folder under `com/pms/` is a business subject (project, billing, workload, governance, mission, agile, kpi, auth, user, team), and every one of them has the same five sub-folders. That repeat is the layered architecture that you can see on disk: `controller/` answers HTTP, `service/` decides, `repository/` talks to the database, `entity/` is the shape of a table row, and `dto/` + `mapper/` translate between inside and outside. `shared/` is the only folder that is not a business subject: it holds what every module needs (security setup, the audited `BaseEntity`, the global error handler). `db/migration/` holds 28 numbered SQL files — the database is built by playing them again in order, never by hand. The same idea comes back on the frontend: `features/` copies the backend modules almost name for name, `core/` holds the things used everywhere (login service, guards, the HTTP interceptor), and `shared/` holds reusable widgets. 192 Java files and 66 TypeScript files sit inside this skeleton.

```
10 @Injectable({ providedIn: 'root' })
11 export class ProjectService {
12     private readonly API = `${environment.apiUrl}/projects`;
13
14     constructor(private http: HttpClient) {}
15
16     list(page = 0, size = 20, sort = 'code,asc'): Observable<PagedResponse<Project>> {
17         const params = new HttpParams()
18             .set('page', page).set('size', size).set('sort', sort);
19         return this.http.get<PagedResponse<Project>>(this.API, { params });
20     }
```

What this does. `@Injectable({ providedIn: 'root' })` tells Angular to build exactly one copy of this class for the whole application and give it to any screen that asks. That is dependency injection, and it means you never write `new ProjectService()` yourself. `private readonly API` builds the address once from `environment.apiUrl`, so moving between the development server and the Docker deployment is a one-line change in one file. `constructor(private http: HttpClient)` gets Angular's HTTP client the same way. `list(...)` has default values (`page = 0, size = 20`), so a caller can just write `list()`. `Observable` means the answer is not ready yet: the method returns a stream, a bit like a promise, that will give you the page of projects when the network answers, and the screen subscribes to it. Now look at what is not there — no margin, no filtering by role, no rule. The browser asks; the server decides.

```
11     const token = auth.getAccessToken();
12     const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
13
14     return next(authReq).pipe(
15       catchError((err: HttpErrorResponse) => {
16         // H-2: server-side first-login gate returned 403 with FIRST_LOGIN_REQUIRED
17         if (err.status === 403 && err.error?.code === 'FIRST_LOGIN_REQUIRED') {
18           router.navigate(['/change-password']);
19           return throwError(() => err);
20         }
21
22         const isAuthEndpoint = req.url.includes('/auth/refresh') ||
23 req.url.includes('/auth/logout');
24         if (err.status === 401 && !isAuthEndpoint) {
25           // H-1: handleTokenRefresh() ensures only one /auth/refresh call goes out at a time.
26           // Concurrent 401 callers subscribe to the shared BehaviorSubject and all get
27           // the same refreshed token without firing duplicate refresh requests.
28           return auth.handleTokenRefresh().pipe(
29             switchMap(res => {
30               const retried = req.clone({ setHeaders: { Authorization: `Bearer ${res.accessToken}` }
31             }));
32
33             return next(retried);
34           }
35         }
36       })
37     );
```

What this does. This function runs for every single HTTP request the application makes, so the token code is written once instead of in fourteen services. `req.clone({ setHeaders: ... })` is needed because an Angular request object cannot be changed — you cannot edit it, so you make a copy with the extra header. `next(authReq)` sends it on. `catchError` then looks at the failures. A 403 carrying the code `FIRST_LOGIN_REQUIRED` means the backend said no because the user still has his temporary password, so the interceptor pushes him to the change-password screen instead of showing an error that means nothing to him. A 401 means "the token has expired": the interceptor calls `handleTokenRefresh()`, then builds the *same* request again with the new token and sends it once more. That is why the user never notices that access tokens only live 15 minutes. The check `!isAuthEndpoint` stops an endless loop where a failing refresh call would try to refresh itself.

```
20 @Tag(name = "Projets", description = "CRUD projets, cycle de vie, fiche d'identification,
21 archivage")
22 @RestController
23 @RequestMapping("/api/projects")
24 @RequiredArgsConstructor
25 public class ProjectController {
26
27     private final ProjectService projectService;
28
29     @GetMapping
30     public ResponseEntity<Page<ProjectResponse>> list(
31         @PageableDefault(size = 20, sort = "code", direction = Sort.Direction.ASC) Pageable
32         pageable) {
33         return ResponseEntity.ok(projectService.findAll(pageable));
34     }
35 }
```

What this does. `@RestController` means "every method here answers an HTTP call, and what it returns is turned into JSON by itself". `@RequestMapping("/api/projects")` is the shared start of the address, so `@GetMapping` with no path means `GET /api/projects`. `@RequiredArgsConstructor` is a Lombok shortcut that writes the constructor for the one `final` field, and Spring then hands in the real `ProjectService` when the app starts. `@PageableDefault` gives you pagination for free: if the browser sends nothing, the server returns 20 projects sorted by code. The body of the method is one line — it passes the call to the service and wraps the answer in HTTP 200. That emptiness is on purpose, and it is the rule written in [docs/ARCHITECTURE.md §4.1](#): "controllers never touch repositories directly" and they carry no business logic. Because of that, the same rules apply whether the call comes from Angular, from Swagger, or from a mobile app built later.

```
49     @PreAuthorize("hasAuthority('VIEW_PROJECT')")
50     @Transactional(readOnly = true)
51     public Page<ProjectResponse> findAll(Pageable pageable) {
52         if (scopeService.hasAllAccess()) {
53             return projectRepository.findAllActivePaged(pageable).map(this::toResponse);
54         }
55         Set<Long> accessible = scopeService.accessibleProjectIds(currentEmail());
56         if (accessible.isEmpty()) return Page.empty(pageable);
57         return projectRepository.findAllActiveByIdIn(accessible, pageable).map(this::toResponse);
58     }
```

What this does. This tiny method holds the two security decisions the whole platform is built on.

`@PreAuthorize("hasAuthority('VIEW_PROJECT')")` asks "does this user have the capability `VIEW_PROJECT`?" — it asks about a permission, never about a role name. That is decision ADR-001: moving a responsibility from one role to another becomes one row in the database, not a change in the code. The second decision is the data scope (ADR-021): having the permission tells you *what* you may do, not *on which projects*. So if the user is not a full-access profile, `accessibleProjectIds(...)` returns the ids he is allowed to see, and the query is kept to those ids only. `if (accessible.isEmpty()) return Page.empty(pageable)` is small but real: asking SQL for `id IN ()` with an empty list would break. `@Transactional(readOnly = true)` opens a read-only database transaction, which lets the driver skip the extra work it does for writes. Last, `.map(this::toResponse)` turns each entity into a DTO, and inside `toResponse` it removes every money field when the caller does not have `VIEW_KPI`. That is business rule BR-050, the money wall around developers.

```

45     /**
46      * M-6 : requête unique remplaçant les 3 appels séparés de ProjectScopeService
47      * (user lookup + chef-projects + team-assignments).
48      * Retourne les IDs de tous les projets actifs dont l'utilisateur est chef de projet
49      * OU membre actif d'équipe.
50      */
51     @Query("SELECT DISTINCT p.id FROM Project p " +
52           "LEFT JOIN p.chefProjet cp " +
53           "WHERE p.deleted = false AND p.archived = false " +
54           "AND (cp.email = :email " +
55           "OR EXISTS (SELECT ta FROM TeamAssignment ta " +
56           "                WHERE ta.project = p AND ta.user.email = :email AND ta.deleted = false))")
57     Set<Long> findAccessibleProjectIdsByEmail(@Param("email") String email);

```

What this does. A repository in Spring Data JPA is an interface with no body: you write the method name, and the framework writes the code for it when the app starts. `@Query` lets you write the search yourself in JPQL — a query language that looks like SQL but names Java classes and fields (`Project p` , `p.chefProjet`) instead of tables and columns. This one answers the question "which projects may this email address see?" in one single trip to the database: the projects he leads (`cp.email = :email`), plus the ones where an active `TeamAssignment` row links him to the project. `:email` is a named parameter, filled in safely by the driver, and that is what makes SQL injection impossible here. Two filters come back everywhere in the codebase: `deleted = false` and `archived = false`. Nothing is ever really erased (decision ADR-009, soft delete), so every query has to leave out the rows marked deleted on purpose. The comment says why the method is there: it replaced three separate trips to the database with one.

Extra explanations

SECTION 3 SAYS A BUSINESSRULEEXCEPTION BECOMES "AN HTTP 409 CONFLICT". THAT IS WRONG. IN BACKEND/SRC/MAIN/JAVA/COM/PMS/SHARED/EXCEPTION/GLOBALEXCEPTIONHANDLER.JAVA, THE HANDLER FOR BUSINESSRULEEXCEPTION RETURNS HTTPSTATUS.UNPROCESSABLE_ENTITY, WHICH IS 422. 409 IS USED FOR ILLEGALARGUMENTEXCEPTION AND FOR DATAINTEGRITYVIOLATIONEXCEPTION. A JURY MEMBER WHO OPENS THE FILE WILL CATCH THIS.

When a business rule is broken, the service throws a `BusinessRuleException`. An example is when the milestone percentages would add up to more than 100. A class marked `@RestControllerAdvice`, called `GlobalExceptionHandler`, catches every exception thrown anywhere in the backend and turns it into an HTTP answer. So the rules for turning errors into codes live in one file, and not in every controller. For a `BusinessRuleException` it answers HTTP 422 (Unprocessable Entity). That means "I understood your request, it was well built, but I say no because of a rule". Here are the other codes it gives. 400 for a field with a bad shape (`MethodArgumentNotValidException`, raised by `@Valid`). 401 for a bad password or a token that is not valid. 403 for a permission you do not have, or for a project outside your scope (`AccessDeniedException`). 404 for `NotFoundException`. 409 for a duplicate row that hits a database rule (`DataIntegrityViolationException`). And 429 when too many login tries were made. The

body of the answer is a ProblemDetail, the standard Spring error shape: {"type", "title", "status", "detail"}.

THE AUTH SECTION QUOTES READONLY VISIBLESECTIONS = COMPUTED(...), AND THE BILLING COMPONENT IS FULL OF SIGNAL() AND COMPUTED(). BUT "SIGNAL" IS NEVER DEFINED, SO THE READER IS LEFT THINKING THAT ANGULAR REDRAWS THE PAGE BY MAGIC.

Angular 20 keeps the values that change in signals. A signal is a box that holds one value. You make it with signal(initialValue). You read it by calling it like a function, this.jalons(). And you put new content in it with this.jalons.set(newList). A computed() is a second box whose value is worked out from other signals, and Angular works it out again by itself as soon as one of them changes. The important result is that you never have to tell the template to refresh. The moment billing.component.ts does this.jalons.set(d) inside reload(), every part of the HTML that reads jalons() is drawn again: the table rows, the empty state and the total. It is the same idea in the sidebar. visibleSections is computed from auth.permissions(), so when the permission list changes the menu redraws by itself. This takes the place of the older Angular way, where the framework had to check the whole page again after every event to guess what had changed.

@TRANSACTIONAL IS DEFINED IN A GENERAL WAY, AS "EVERYTHING SUCCEEDS TOGETHER OR IS UNDONE TOGETHER". BUT IT IS NEVER MADE CONCRETE WITH A PMS EXAMPLE, SO THE READER CANNOT SAY WHAT WOULD REALLY BE UNDONE.

@Transactional above a service method means this. Open one database transaction when the method starts. Commit it if the method ends normally. Undo everything if it throws. Let us make it concrete with amendments. When a client signs an avenant (a signed change to the contract), AvenantService saves the amendment row, raises the revisedBudget of the project, and then asks JalonService.recomputePrevuMontants(project) to price every milestone still in status PREVU again. That is three writes. If the third one failed halfway and there were no transaction, you would end up with an amendment recorded, a budget raised, and half the milestones priced on the old budget. That is exactly the kind of quiet mismatch the Excel workbook used to produce. Because the whole method runs inside one transaction, PostgreSQL either applies all three or none of them. There is also a read-only version, @Transactional(readOnly = true). It is used on methods such as JalonService.findByProject that only read. It tells the driver that no write bookkeeping is needed, and it makes a write by accident fail loudly.

ONLY FOUR OF THE SEVEN SECTIONS COULD BE READ IN THE MATERIAL I WAS GIVEN. THE FOURTH ONE WAS CUT OFF IN THE MIDDLE OF A SENTENCE, AT 'SNAPSHOT_KPIS — A DATED', AND THREE MORE SECTIONS NEVER CAME AT ALL. SO THE READING ORDER AND THE LINKS BETWEEN SECTIONS THAT I PROPOSE COVER FOUR CHAPTERS ONLY. AND THE GUIDE STILL HAS NO PAGE THAT SAYS "HOW THE CHAPTERS CONNECT".

Add a short first page called "How to read this guide", with the map of the chapters and one sentence on each about how it hands over to the next one. The architecture chapter shows the round trip. The database chapter explains what the round trip writes. The auth chapter explains who is allowed to

make it. The financial chapter explains what the stored numbers are turned into. Then say the three cross-links out loud, because a beginner misses exactly these. (1) The permission text 'MANAGE_BILLING' is the very same text in three places: a row in the permissions table, a @PreAuthorize on a Java method, and an if in an Angular template. (2) A URL under /api/projects/{id}/ is checked twice: by ProjectScopeInterceptor for the project, and by @PreAuthorize for the capability. (3) Every money figure on a KPI screen is worked out again from rows written by other chapters. The days come from the workload module, the rates from the resources module, and the percentages from the billing module. Nothing is stored already computed, except the dated snapshots.

Words used in this chapter

Tier	One of the three pieces of the system that run on their own — the browser application, the server application, and the database — where each one can be restarted, replaced or grown without rewriting the other two.
REST API	The set of web addresses the backend answers, where the address says what you want (/api/projects/7) and the HTTP verb says what to do with it (GET to read, POST to create, PUT to replace, PATCH to change one thing, DELETE to remove).
Layered architecture	A rule that the code inside one module is stacked in fixed layers — controller (HTTP), service (decisions), repository (database access), entity (table shape) — and a layer may only call the one right below it.
Modular monolith	One single program you deploy, but cut inside into feature modules that stand on their own, chosen in ADR-016 instead of microservices because one team, one database and figures about money that always agree are easier to build, to test and to defend.
Entity	A Java class such as <code>Project</code> whose fields match the columns of a database table one for one, so saving the object writes the row.
DTO (Data Transfer Object)	A plain object shaped for the outside world, like <code>ProjectResponse</code> , used so entities never leave the service layer and so the JSON sent to the browser can hide fields such as the budget.
JWT (JSON Web Token)	A short signed piece of text the browser sends with every request to prove who the user is, so the server does not have to keep a session in memory for each connected user.
Authority / permission	A capability code such as <code>VIEW_PROJECT</code> or <code>MANAGE_DI</code> that is given to a role in the database and checked by the code; the code never checks a role name.

Flyway migration	A numbered SQL file (<code>v5__schema_project.sql</code>) that Flyway runs once and only once, in order, so any empty PostgreSQL instance can be rebuilt into exactly the schema you expect — 28 of them exist today.
Reverse proxy	A server, here nginx, that sits in front and decides for each request coming in whether to answer it itself (the Angular files) or pass it to another server (<code>/api/...</code> goes to Spring Boot).

Traps to avoid

You might think the frontend hides the budget from a developer by not showing it, but actually the backend never sends it at all: `ProjectService.toResponse()` calls `response.withoutFinancials()` whenever the caller does not have `VIEW_KPI`. Hiding a field in the browser would be only decoration — someone could open the network tab and read it. The Angular guards and the menu that changes with your rights are there only for comfort. The real wall is on the server.

You might think the three Docker containers are three separate applications talking over the internet, but actually they are one application in three boxes on a private network, and only nginx can be reached from your machine. `docker-compose.yml` publishes the frontend on port 8081. The backend answers on `http://backend:8080` — a name that only exists inside the `pms-net` network. This is why it matters that nginx passes `/api` on: it makes the app and the API share one origin, so no CORS setup is needed and the `SameSite=Strict` refresh cookie works with no trouble.

You might think `ProjectRepository` is an unfinished file because its methods have no body, but actually that is the whole point: Spring Data JPA reads the interface when the app starts and writes the code for it. `existsByCodeAndDeletedFalse(String code)` needs no code at all — the method name is the query. You only write `@Query` when the question is too hard to say as a name.

You might think deleting a project takes it out of the database, but actually `delete()` only calls `project.setDeleted(true)` and saves. Every query then adds `WHERE p.deleted = false`. That is decision ADR-009 (soft delete): the old charge records, the assignments, the TCC rates and the billing records must stay for audit, so nothing about money is ever really erased.

You might think the ten modules (project, billing, workload, governance, mission, agile, kpi, auth, user, team) are ten separate programs, but actually they compile into one JAR file and share one database and one transaction. They are split for human reasons — you can open `billing/` and read everything about invoicing in one place. ADR-016 keeps the split honest with one rule: a module may call another module's *service*, never another module's repository. That is why `ProjectService` imports `JalonService` from the billing module and not a billing repository.

Database schema and Java-to-table mapping (PostgreSQL + Flyway migrations + JPA/Hibernate entities)

This is the part of PMS that keeps everything for good: it says what the filing cabinets (the tables) inside PostgreSQL look like, it keeps their shape under version control so every copy of the application has the same shape, and it turns Java objects into rows of those tables and back.

THINK OF IT LIKE...

Think of a restaurant that keeps paper registers. Each register is a table: one for staff, one for contracts, one for the days worked. Every sheet in a register has a printed number in the corner, and that number is the primary key. A sheet in the "days worked" register also carries the contract number of the job it belongs to, and that is the foreign key, so you can always find the right contract. JPA/Hibernate is the clerk. He takes a Java object in his hand and copies it onto the right sheet in the right register, and he reads a sheet back into an object when you ask. Flyway is the binder of dated instruction notes. It says how the registers were built, note by note, so a brand-new branch can build exactly the same registers in the same order.

WHY THIS EXISTS

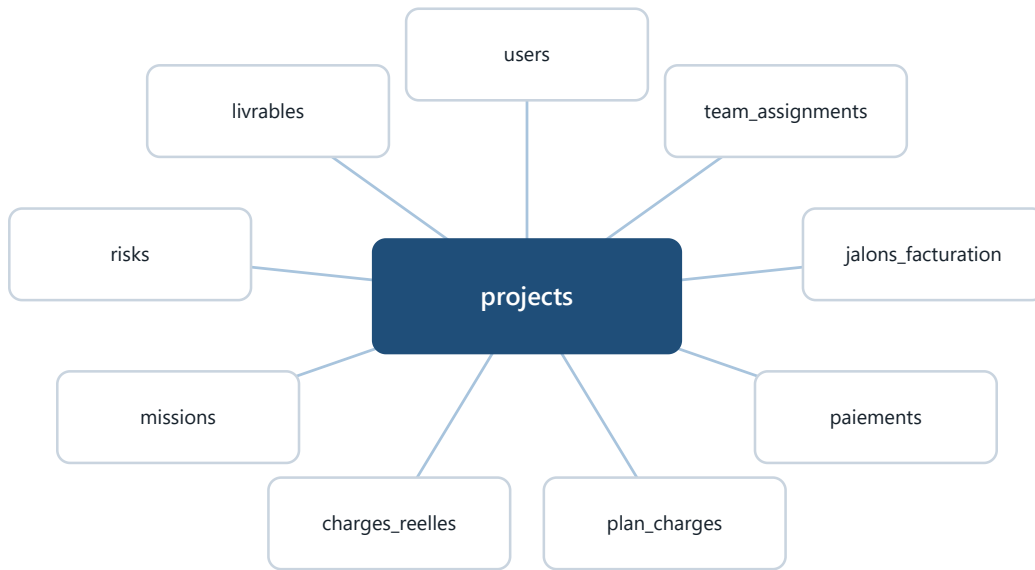
ST2i managed project margins in an Excel workbook. A workbook has real problems. Only one person can edit it safely. There is no login, so everybody sees the salaries and the margins. A wrong formula spreads without a sound. And nothing writes down who changed what. Moving the same information into a relational database — a database that keeps data in tables and links them together — fixes all four problems. Several people can write at the same time, because the database puts the writes in order. Each table can be read only by users with the right permission. The database itself refuses impossible values, like a milestone percentage above 100, or an end date before a start date. And every row carries `created_at` / `created_by` / `updated_at` / `updated_by`. The sheets of the workbook became these tables.

MAIN TABLES, one line each:

- roles — the four job families (ADMIN, DIRECTEUR, CHEF_PROJET, DEVELOPPEUR), plus any role an admin creates.
- permissions — the single things the app can allow, written as codes like VIEW_BILLING or MANAGE_DI.

- `role_permissions` — the tick-box grid that says which role holds which permission (only a link table).
- `users` — the people who log in: name, email, hashed password, their role, and a `token_version` counter used to kill old sessions.
- `resources` — the billable side of a person: daily cost and TCC rate (*taux de charges*, the overhead percentage) and staffing dates.
- `tcc_annuels` — a per-year override of a resource's daily rate and TCC rate, because one day in 2024 does not cost the same as one day in 2025.
- `parameters` — small global settings (default TCC rate, currency, start of the fiscal year).
- `projects` — the heart of the app: code, client, budget, currency and exchange rate, contract type, sold workload in man-days, director and project manager.
- `lignes_di` — the lines of the *Devis Interne* (the internal quote): fees, expenses, taxes. Structure only; real company figures are never seeded.
- `team_assignments` — who is on which project team, with a role in the team and a date range.
- `plan_charges` — planned man-days: one row per project, per person, per month.
- `charges_reelles` — the man-days really worked: the same shape as `plan_charges`, plus who validated them and when.
- `jalons_facturation` — the billing milestones: a label, a percentage of the budget, the amount that comes out of it, and a status `PREVU/FACTURE/PAYE`.
- `paiements` — money really received against a milestone.
- `avenants` — the contract amendments that change the budget and/or the sold workload.
- `missions` — professional trips tied to a project and to a person.
- `composantes_mission` — the cost lines of a trip (per diem, plane ticket, transport, stay).
- `risks` — project risks with probability, impact, mitigation plan and status.
- `livrables` — the deliverables with a due date and a status.
- `parties_prenantes` — the stakeholders with their level of influence and interest.
- `demandes_changement` — the change requests with a requester, a priority and a decision date.
- `snapshot_kpis` — a dated photograph of a project's indicators (consumed budget, EAC, margin, EVM figures) so trends can be drawn.
- `sprints` and `backlog_items` — the agile planning module: the iterations and the work items inside them.
- `flyway_schema_history` — created by Flyway itself; the list of the migrations already applied and their checksums.

The picture



The projects table and its neighbours — Notice that projects sits in the middle. Every other box carries a `project_id` that points back at it. `users` is the only other table that many boxes point to. That is why really deleting a user or a project would break links everywhere, and that is why PMS soft-deletes instead.

Step by step

1

The shape of the database is written by hand, in SQL files

Nobody lets the framework invent the tables. The folder `backend/src/main/resources/db/migration/` holds 28 plain SQL files, V1 to V28. V1 creates the login tables. V5 creates projects. V7 creates the two workload tables. V9 creates the billing tables. V11 creates the governance tables. V27 creates the agile tables. Writing the SQL by hand means the author picks the exact column types, the exact rules, and the exact indexes.

```
FILES backend/src/main/resources/db/migration/V1__schema_auth.sql ·
      backend/src/main/resources/db/migration/V5__schema_project.sql ·
      backend/src/main/resources/db/migration/V9__schema_billing.sql
```

2

Flyway runs those files in number order, one time each, and remembers

Flyway is a small tool that runs when the application starts. It looks inside that folder, sorts the files by their version number, and runs any file it has not run before on this database. Then it writes a row into its own table, `flyway_schema_history`. That row holds the version, the file name and a checksum, which is a fingerprint of the text of the file. At the next startup it sees that V1 to V28 are already written down, so it does nothing. Starting the app twice never creates the tables twice.

```
FILES backend/src/main/resources/application.yml
```

3

A migration is never edited after it has run — a mistake gets a new file

Flyway keeps a fingerprint of each file. So if you change V5 after it has run, the fingerprint no longer matches and Flyway refuses to start (in the prod profile, where `validate-on-migrate` is on). And there is something even more important. Editing V5 would fix nothing on a server where V5 already ran months ago. So corrections are always added as a new numbered file. V17 is exactly that: it drops the date rule created in V5 and adds a corrected one. V20 deleted the `MANAGE_ROLES` permission when that feature was dropped, and V25 put it back later when the feature came back. Two files, and neither one was edited.

```
FILES backend/src/main/resources/db/migration/V17__fix_project_date_check.sql ·
      backend/src/main/resources/db/migration/V20__descope_unused_permissions.sql ·
      backend/src/main/resources/db/migration/V25__rbac_admin_module.sql
```

4

Each table gets one matching Java class, called an entity

An entity is a normal Java class marked with the annotation `@Entity`. That annotation tells JPA that one object of this class is one row of a table. `@Table(name = "jalons_facturation")` names the table it belongs to. Each field with `@Column` is one column. So `JalonFacturation.java` and the `jalons_facturation` table are two views of the same thing: one that Java code can hold in memory, one that PostgreSQL can keep on disk.

```
FILES backend/src/main/java/com/pms/billing/entity/JalonFacturation.java ·
      backend/src/main/java/com/pms/project/entity/Project.java ·
      backend/src/main/java/com/pms/user/entity/User.java
```

5

The columns every table shares are written one time, in BaseEntity

Every table in PMS has the same six housekeeping columns: `id`, `created_at`, `updated_at`, `created_by`, `updated_by`, `deleted`. Instead of writing them again in 24 classes, they live in one parent class, `BaseEntity`, marked `@MappedSuperclass`. That annotation means: I am not a table of my own, but every class that extends me gets my columns. `Project`, `User`, `Mission` and all the others just say extends `BaseEntity`. The `created_by` and `updated_by` values are filled in by themselves, from the user who is logged in, by `SpringSecurityAuditorAware`. When nobody is logged in, it falls back to the plain word "system".

```
FILES backend/src/main/java/com/pms/shared/entity/BaseEntity.java ·
      backend/src/main/java/com/pms/shared/config/SpringSecurityAuditorAware.java ·
      backend/src/main/java/com/pms/shared/config/JpaConfig.java
```

6

Links between tables become object references, written with @ManyToOne

In SQL, a billing milestone points at its project through a `project_id` column, and that value must match an existing `projects.id`. That is a foreign key. In Java, the same link is written as a field of type `Project` with the annotation `@ManyToOne`, which you can read word for word: many milestones belong to one project. `@JoinColumn(name = "project_id")` names the column that holds the link. So Java code writes `jalon.getProject().getName()` and does not run a second query by hand. `Project` itself has two such links back to users: `director` and `chefProjet`, and both point at the users table.

```
FILES backend/src/main/java/com/pms/billing/entity/JalonFacturation.java ·
      backend/src/main/java/com/pms/project/entity/Project.java ·
      backend/src/main/java/com/pms/workload/entity/ChargeReelle.java
```

7

Repositories turn Java method calls into SQL

A repository is a Java interface that extends `JpaRepository`. Spring Data writes the code behind it for you at startup. Some methods have a name so plain that the name itself is the query. `existsByCodeAndDeletedFalse(String code)` becomes `SELECT ... WHERE code = ? AND deleted = false`. Other methods carry an explicit `@Query` written in JPQL. JPQL is a query language that looks like SQL, but it names Java classes and fields instead of tables and columns. Notice that every single read in `ProjectRepository` ends with `deleted = false`.

```
FILES backend/src/main/java/com/pms/project/repository/ProjectRepository.java ·
      backend/src/main/java/com/pms/billing/repository/JalonFacturationRepository.java
```

8

Deleting never takes a row away: it flips the deleted flag (soft delete)

In all 19 delete operations in the backend, the service loads the row, calls `setDeleted(true)` and saves it. For example, `project.setDeleted(true)` in `ProjectService.delete`. The row stays in PostgreSQL for ever. So a milestone that was already invoiced can still be followed, and the foreign keys that point at it are never broken. The price is that every read must remember to filter `deleted = false`. The other price is that plain UNIQUE constraints get in the way — and that is why V18 replaced them with partial unique indexes that watch only the live rows.

FILES `backend/src/main/java/com/pms/project/service/ProjectService.java` ·
`backend/src/main/java/com/pms/billing/service/JalonService.java` ·
`backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql`

The real code

`backend/src/main/resources/db/migration/V1__schema_auth.sql`

SQL

```

38 CREATE TABLE users (
39     id          BIGSERIAL    PRIMARY KEY,
40     first_name  VARCHAR(100) NOT NULL,
41     last_name   VARCHAR(100) NOT NULL,
42     email       VARCHAR(255) NOT NULL,
43     password_hash VARCHAR(255) NOT NULL,
44     active      BOOLEAN      NOT NULL DEFAULT TRUE,
45     first_login  BOOLEAN      NOT NULL DEFAULT TRUE,
46     token_version INT         NOT NULL DEFAULT 0,
47     role_id     BIGINT        NOT NULL,
48     created_at  TIMESTAMP     NOT NULL DEFAULT NOW(),
49     updated_at  TIMESTAMP     NOT NULL DEFAULT NOW(),
50     deleted     BOOLEAN      NOT NULL DEFAULT FALSE,
51     CONSTRAINT uk_users_email UNIQUE (email),
52     CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles(id)
53 );

```

What this does. `CREATE TABLE users` makes one filing register. Each line inside it is one column, that is one box that every sheet in that register has. `id BIGSERIAL PRIMARY KEY` does two jobs. `BIGSERIAL` means PostgreSQL picks the next whole number by itself (1, 2, 3 ...), so nobody has to invent an identifier. `PRIMARY KEY` means that number names the row, and it can never repeat and never be empty. `NOT NULL` on `first_name` means the database refuses a row with no first name, not just the app. The column keeps `password_hash`, not the password: the real password is never written down. `role_id BIGINT` with `CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles(id)` is the link to another table. A foreign key is a column whose value must already exist as a primary key in the table it names, so PostgreSQL will refuse a user who points at role 99 if role 99 does not exist. `UNIQUE (email)` forbids two users with the same email. And `deleted BOOLEAN DEFAULT FALSE` is the soft-delete switch that every table in PMS carries.

```
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     private Long id;
25
26     @CreatedDate
27     @Column(name = "created_at", nullable = false, updatable = false)
28     private LocalDateTime createdAt;
29
30     @LastModifiedDate
31     @Column(name = "updated_at", nullable = false)
32     private LocalDateTime updatedAt;
33
34     @CreatedBy
35     @Column(name = "created_by", updatable = false, length = 255)
36     private String createdBy;
37
38     @LastModifiedBy
39     @Column(name = "updated_by", length = 255)
40     private String updatedBy;
41
42     @Column(nullable = false)
43     private boolean deleted = false;
```

What this does. This block sits inside BaseEntity, a class marked @MappedSuperclass on line 15. That annotation means: I am not a table myself, but every class that extends me gets these columns. @Id marks the field that is the primary key. @GeneratedValue(strategy = GenerationType.IDENTITY) tells Hibernate not to invent the number, but to let PostgreSQL's BIGSERIAL make it and then read it back. @Column(name = "created_at") is the bridge between two naming habits: Java writes createdAt, SQL writes created_at, and this annotation says they are the same thing. updatable = false is a real rule, not decoration. Once a row is written, Hibernate will never put created_at or created_by in a later UPDATE, so the creation stamp cannot be rewritten. @CreatedDate, @LastModifiedDate, @CreatedBy and @LastModifiedBy are filled in by themselves, because JpaConfig switches on @EnableJpaAuditing. The last field, deleted, starts at false and is the one flag that says whether a row is still alive.

```
11 @Entity
12 @Table(name = "jalons_facturation")
13 @Getter
14 @Setter
15 @NoArgsConstructor
16 @AllArgsConstructor
17 @Builder
18 public class JalonFacturation extends BaseEntity {
19
20     @ManyToOne(fetch = FetchType.LAZY)
21     @JoinColumn(name = "project_id", nullable = false)
22     private Project project;
23
24     @Column(nullable = false, length = 255)
25     private String label;
26
27     @Column(nullable = false, precision = 5, scale = 2)
28     private BigDecimal pourcentage;
```

What this does. `@Entity` is the line that turns a normal Java class into a table mapping: one object is one row. `@Table(name = "jalons_facturation")` says which table, because here the class name and the table name are not the same. `extends BaseEntity` is how this class quietly gets `id`, `created_at`, `updated_at`, `created_by`, `updated_by` and `deleted`. `@ManyToOne` is the Java way of writing a foreign key, and you can read it word for word: many milestones belong to one project. `@JoinColumn(name = "project_id", nullable = false)` names the column that holds the link and says it can never be empty, so a milestone without a project cannot exist. `fetch = FetchType.LAZY` means Hibernate does not load the whole Project at once. It gives back a stand-in object, and it runs the second query only if the code really touches a project field. The percentage is a `BigDecimal`, not a `double`, with `precision = 5`, `scale = 2`. That means at most five digits, and two of them after the point. Money and percentages must never be kept as floating-point numbers, because in a `double` `0.1 + 0.2` is not exactly `0.3`, and a margin report would slowly drift away.

```
1  -- =====
2  -- V17 : Correction de la contrainte chk_project_dates
3  -- =====
4  -- La règle métier (modèle Excel, bornes incluses) autorise un
5  -- projet d'un seul jour (start_date = end_date, durée = 1 j).
6  -- Ancienne contrainte : end_date > start_date (strict - 500 sur projet 1 jour)
7  -- Nouvelle contrainte : end_date >= start_date (inclus)
8
9  ALTER TABLE projects DROP CONSTRAINT chk_project_dates;
10 ALTER TABLE projects ADD CONSTRAINT chk_project_dates
11     CHECK (end_date IS NULL OR end_date >= start_date);
```

What this does. This tiny file is the clearest proof of the migration rule. V5 had created projects with CHECK (end_date IS NULL OR end_date > start_date). A check constraint is a condition that PostgreSQL tests on every insert and every update, and it refuses the row if the condition is false. That strict "greater than" made a project of one day impossible, and the server answered with an error 500. The fix was NOT to open V5 and change the sign, because V5 has already run on every existing database, and running it again is not an option. Instead, a new file, V17, drops the old constraint by its name and adds a corrected one with >=, so a start date equal to the end date is accepted. That matches the Excel habit, where both ends count. So the history of the database reads like a story: the rule was created in V5 and corrected in V17. Any fresh database plays the same story again and lands in the same final state.

```
4  -- Les contraintes UNIQUE absolues sur users.email et projects.code
5  -- bloquaient la réutilisation de ces valeurs après un soft-delete.
6  -- Remplacées par des index uniques partiels (WHERE deleted = FALSE)
7  -- pour n'interdire les doublons que parmi les enregistrements actifs.
8
9  -- — users.email —————
10 ALTER TABLE users DROP CONSTRAINT uk_users_email;
11 CREATE UNIQUE INDEX uk_users_email ON users(email) WHERE deleted = FALSE;
12
13 -- — projects.code —————
14 ALTER TABLE projects DROP CONSTRAINT uk_projects_code;
15 CREATE UNIQUE INDEX uk_projects_code ON projects(code) WHERE deleted = FALSE;
```

What this does. This migration exists because soft delete and plain uniqueness fight each other. V1 said that email must be unique in the whole users table. But a soft-deleted user keeps its row. So hiring the same person again with the same email, or using a project code again after cancelling a project, was refused by the database. The fix replaces each UNIQUE constraint with a partial unique index. An index is a sorted lookup structure that PostgreSQL keeps next to the table, and this one covers only the rows that match WHERE deleted = FALSE. In plain words: among the rows still alive, an email or a project code may appear only one time; among the dead rows, copies are allowed and nobody cares. The same pattern comes back all over the schema. uk_pc_active on plan_charges(project_id, user_id, period) forces one planned-workload line per person, per project, per month — but only among the live rows, so a deleted line can be typed in again.

```

16 public interface ProjectRepository extends JpaRepository<Project, Long> {
17
18     @Query("SELECT p FROM Project p LEFT JOIN FETCH p.director LEFT JOIN FETCH p.chefProjet WHERE
19 p.deleted = false AND p.archived = false ORDER BY p.code")
20     List<Project> findAllActive();
21
22     @Query("SELECT p FROM Project p LEFT JOIN FETCH p.director LEFT JOIN FETCH p.chefProjet WHERE
23 p.deleted = false AND p.archived = true ORDER BY p.code")
24     List<Project> findAllArchived();
25
26     @Query("SELECT p FROM Project p LEFT JOIN FETCH p.director LEFT JOIN FETCH p.chefProjet WHERE
27 p.id = :id AND p.deleted = false")
28     Optional<Project> findActiveById(Long id);
29
30     @Query("SELECT p FROM Project p LEFT JOIN FETCH p.director LEFT JOIN FETCH p.chefProjet WHERE
31 p.chefProjet.id = :userId AND p.deleted = false")
32     List<Project> findActiveByChefProjetId(Long userId);
33
34     boolean existsByCodeAndDeletedFalse(String code);
35
36     Optional<Project> findByCodeAndDeletedFalse(String code);

```

What this does. A repository is an interface with no body. Spring Data reads it at startup and writes the code behind it, so no SQL is written by hand in Java code. Two styles sit side by side here. The last two methods carry no query at all: the name of the method IS the query. `existsByCodeAndDeletedFalse` becomes `SELECT count(*) ... WHERE code = ? AND deleted = false`, and that is how a duplicate project code is found before an insert. The others carry an explicit `@Query` in JPQL. JPQL names the Java class `Project` and the Java field `chefProjet`, not the table `projects` and the column `chef_projet_id`, and Hibernate turns it into real SQL. `LEFT JOIN FETCH p.director` is the cure, done on purpose, for lazy loading. It orders Hibernate to bring back the director and the project manager inside the same single query, instead of firing one extra query per project when the code reads their names later. That is the classic N+1 problem. Every read filters `p.deleted = false`, and the first two also keep archived apart from active. Archived is a business state that means "finished, hide it from the working list", and that is not the same thing as deleted.

Extra explanations

THE DATABASE SECTION LISTS `CREATED_AT` / `CREATED_BY` / `UPDATED_AT` / `UPDATED_BY` AND SAYS "EVERY CHANGE HAS A NAMED AUTHOR". BUT IT NEVER SHOWS THE MACHINERY THAT FILLS THOSE COLUMNS, SO A BEGINNER THINKS EACH SERVICE WRITES THEM BY HAND.

Nobody writes the audit columns by hand. Every table class in PMS extends `backend/src/main/java/com/pms/shared/entity/BaseEntity.java`, which holds the id, the four audit fields and the deleted flag in one place. `@MappedSuperclass` means "these fields are copied into every table that inherits this class", so the class is not a table of its own. The fields carry `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy` and `@LastModifiedBy`. And `@EntityListeners(AuditingEntityListener.class)` asks Hibernate to fill them just before it writes the row. The feature is turned on by `@EnableJpaAuditing(auditorAwareRef =`

"springSecurityAuditorAware") in backend/src/main/java/com/pms/shared/config/JpaConfig.java. The name of the author comes from backend/src/main/java/com/pms/shared/config/SpringSecurityAuditorAware.java, which reads the logged-in user out of the SecurityContextHolder and returns the text "system" when there is nobody, at start-up seeding for example. So when a Chef de Projet saves a 30% milestone, the row quietly gets created_by = his email and created_at = now. It happens because JalonService built an object whose parent class was already wired for that.

SOFT DELETE IS SHOWN AS "EVERY QUERY MUST ADD DELETED = FALSE", WHICH SOUNDS LIKE A HABIT A DEVELOPER COULD FORGET. THE GUIDE NEVER SAYS WHERE THE FLAG COMES FROM. IT NEVER SAYS THAT THERE IS NO AUTOMATIC GLOBAL FILTER. AND IT NEVER SAYS HOW THIS WORKS TOGETHER WITH THE MILESTONE RULES.

Soft delete means a row is never taken away. A true/false column named deleted is switched to true, and the row stays there for audit. The column is written once, on BaseEntity, so every table in PMS has it. There is no magic here. Hibernate does not hide deleted rows by itself, so each query has to say so out loud. That is why the repository methods are called findActiveById and findActiveByProjectId instead of findById, and why their JPQL always ends with AND j.deleted = false. For example, in backend/src/main/java/com/pms/billing/repository/JalonFacturationRepository.java the percentage total is SELECT COALESCE(SUM(j.pourcentage), 0) ... AND j.deleted = false. So a milestone you deleted stops eating into the 100% budget, while the row itself lives on in PostgreSQL. Deleting is also a business decision, not a technical one. JalonService.delete says no with "Impossible de supprimer un jalon déjà facturé ou payé" when the status is not PREVU. Once an invoice has gone out, accounting is closed and the history has to stay exactly as it was.

THE WORD "RECORD" IS USED FOR EVERY DTO (JALONREQUEST, JALONRESPONSE, LOGINREQUEST, SNAPSHOTREQUEST) WITHOUT BEING SAID OUT LOUD, AND IT IS NEVER DEFINED. MAPSTRUCT IS CALLED "GENERATED AT COMPILE TIME" WITH NO PICTURE OF WHAT THE GENERATED THING DOES. A STUDENT WHO IS ASKED "WHERE IS THE CODE THAT COPIES THE FIELDS?" WOULD HAVE NO ANSWER.

Two small Java ideas carry all the DTOs in PMS. First, a record. Writing `public record JalonRequest(@NotBlank String label, @NotNull @Positive @DecimalMax("100") BigDecimal pourcentage, LocalDate datePrevue) {}` declares a class whose only job is to hold those three values. Java writes the constructor, the read-only accessors, equals and toString for you, and the values cannot be changed once the object is made. That is exactly what you want for a message that comes off the network. Second, MapStruct. backend/src/main/java/com/pms/billing/mapper/JalonMapper.java is an interface with no code in it, marked `@Mapper(componentModel = "spring")`. It declares JalonResponse toResponse(JalonFacturation jalon), plus two renaming rules: `@Mapping(target = "projectId", source = "project.id")` and `@Mapping(target = "projectCode", source = "project.code")`. At compile time MapStruct writes a real class called JalonMapperImpl, and you can open it under backend/target/generated-sources. It copies label to label, montant to montant, and reaches through

project.getId() for projectId. Nothing happens at run time except plain field copies. That is why it is fast, and why a typo in a field name breaks the build instead of production.

THE REPOSITORY QUERIES IN THE GUIDE ARE SHOWN WITH JOIN FETCH, BUT THE TERM IS NEVER EXPLAINED. AND @MANYTOONE(FETCH = FetchType.LAZY) ON JALONFACTURATION.PROJECT IS NEVER MENTIONED AT ALL. THIS IS THE SINGLE MOST LIKELY "WHY DID YOU WRITE IT THAT WAY?" QUESTION AT A SOUTENANCE.

Look at JalonFacturation. The link to its project is written @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "project_id", nullable = false). @ManyToOne means many milestones point at one project. @JoinColumn names the column that holds the project number. LAZY means Hibernate does not load the project when it loads the milestone. It puts a placeholder there and goes to get the project only if somebody really asks for it. That is good for speed, but it has two results. First, if you read fifty milestones and then ask each one for its project code, you get fifty-one queries instead of one. That is the classic "N+1" problem. Second, if you ask after the transaction is closed, Hibernate has no connection left and throws LazyInitializationException. You avoid both by saying up front which links you want, and that is JOIN FETCH. That is why the query in JalonFacturationRepository reads SELECT j FROM JalonFacturation j JOIN FETCH j.project WHERE j.project.id = :projectId AND j.deleted = false. One trip to PostgreSQL brings back the milestones with their project already attached, so JalonMapper can read project.getCode() with no risk.

Words used in this chapter

Relational table	A grid kept inside the database where every column is one named piece of information and every row is one record, like one sheet in a register.
Primary key	The column, here always called id, whose value names one row and only that row, and which is never empty and never repeated.
Foreign key	A column that holds the primary key of a row in another table, so the database itself says no to a link that points at something that is not there.
JPA and Hibernate	JPA is the Java standard that says how a Java class can stand for a database table, and Hibernate is the library that does the real work of writing objects into rows and reading rows back into objects.
Entity	A Java class marked @Entity, where one object in memory is exactly one row of one table.
Flyway migration	A numbered SQL file such as V9__schema_billing.sql that Flyway runs one time only on a database and then writes down, so every environment is built by playing the same files again in the same order.

Soft delete	Deleting by setting a true/false column named <code>deleted</code> to true instead of taking the row away, so the history stays easy to read and nothing that pointed at the row is broken.
Check constraint	A rule attached to a table, such as <code>CHECK (pourcentage > 0 AND pourcentage <= 100)</code> , that PostgreSQL tests on every write and that makes the write fail if it is false.

Traps to avoid

You might think that deleting a project in the interface takes its row out of PostgreSQL, but actually `ProjectService.delete` only runs `project.setDeleted(true)` and saves the object. The row stays in the table forever. It goes off the screen only because every query in `ProjectRepository` ends with `deleted = false`. So if you write a new query one day and forget that filter, deleted projects will come back into the application with no warning at all.

You might think Hibernate creates the tables from the entity classes, but actually `application.yml` sets `ddl-auto: validate`. That means Hibernate is not allowed to change anything. At start-up it only checks that each entity matches a table that already exists. The tables come from the Flyway files. So if you add a field to a Java entity and you do not add a migration, no column is created. The application refuses to start.

You might think `@ManyToOne(fetch = FetchType.LAZY)` means the linked object is always there, but actually Hibernate gives you back an empty stand-in. It goes and gets the real row only when a field is read. And because `open-in-view` is false, that read has to happen inside the transaction. This is exactly why the repository queries say `LEFT JOIN FETCH p.director`. They pull the linked rows in the same query, instead of one extra query for each project.

You might think the rule "billing milestone percentages must not go over 100" is enforced by the database, but actually the `CHECK` in `V9` only keeps each single milestone at 100 or less. The total of all the milestones of one project is checked in Java, in `JalonService.validatePourcentageSum`. That method asks the repository for the current total of the live milestones, adjusts it for the one you are editing, and throws a `BusinessRuleException` if the result would go over 100.

You might think "archived" and "deleted" are the same idea, because both hide a project from the main list, but actually `V16` added archived on purpose to keep the two apart. An archived project is finished, and you can still read all of it in its own section. A deleted project is gone from the application. `findAllActive` filters both. `findAllArchived` filters `deleted = false AND archived = true`.

Authentication and Authorization (login, JWT, permissions, project scope)

This part of PMS checks that you really are the person you say you are when you log in, and then, on every single request, it decides which screens you may open and whose project figures you may see.

THINK OF IT LIKE...

Think of a big conference. At the entrance desk you show your ID one time. The desk checks it and gives you a plastic badge. On the badge there is your name and the list of rooms you may enter. After that, nobody asks for your ID again. The guard at each door only reads the badge. The badge is laminated, so you cannot add a room to the list by yourself. The badge dies after 15 minutes. So you also get a separate paper voucher. You keep it in your pocket and you never show it at the doors. You give it back at the desk and you get a fresh badge. And even inside a room you are allowed to enter, they hand you only the folders that belong to your own team.

WHY THIS EXISTS

Before PMS, ST2i computed project margins in one shared Excel workbook. An Excel file does not know "who you are": anybody who can open the file sees every project, every daily rate, every margin, and can change any cell without a trace. For a consulting firm that is a real business problem. A developer should not read the commercial margin of a project he does not work on, and a Chef de Projet should not read the whole company portfolio. Turning the workbook into a web application only helps if the application can answer two questions on every click. First: "who is this person?" That is authentication. Second: "is this person allowed to do this, on this project?" That is authorization. This subsystem is the answer to both. It is also what gives a meaning to the audit columns (created_by / updated_by), because now every change has an author with a name.

The picture



PMS login sequence: from the form to the signed token — Notice three things. The password leaves the browser only one time. The database is asked only one question, because the user, the role and the permissions all come back

together. And the two tokens come back by two different roads: the access token inside the JSON, and the refresh token inside a cookie that the JavaScript cannot read.

Step by step

1

The browser sends the email and the password one time

On the login page the user types an email and a password, then presses the button. The Angular component calls `AuthService.login()`. That method sends one HTTP POST to `/api/auth/login` with the two values in the body. The option `withCredentials: true` is important: it tells the browser to accept the cookie that the server is about to set, and to send it back later. Note that this URL is one of the very few that Spring Security lets through without a token. `SecurityConfig` lists `/api/auth/login` and `/api/auth/refresh` as `permitAll()`, and it says `.anyRequest().authenticated()` for everything else.

```
FILES frontend/pms-frontend/src/app/features/auth/login/login.component.ts · frontend/pms-frontend/src/app/core/services/auth.service.ts · backend/src/main/java/com/pms/auth/controller/AuthController.java · backend/src/main/java/com/pms/shared/config/SecurityConfig.java
```

2

The server refuses to answer too fast, and it checks the password without giving hints

`AuthService.login()` first asks `LoginAttemptTracker` one question: has this email already failed 5 times in the last 15 minutes? If yes, it throws right away and the user gets HTTP 429 ("too many requests"). Then it loads the user from PostgreSQL. The database never keeps the password itself. It keeps only a bcrypt hash, which is a one-way scramble, so even a person who reads the table cannot get the password back. The code compares the typed password with that hash. Here is the clever detail. When the email does not exist at all, the code still runs bcrypt against a fake hash that was worked out in advance. So a wrong email and a wrong password take exactly the same amount of time. Without that, an attacker could measure the response times and find out which emails are real.

```
FILES backend/src/main/java/com/pms/auth/service/AuthService.java · backend/src/main/java/com/pms/auth/service/LoginAttemptTracker.java · backend/src/main/java/com/pms/shared/config/SecurityConfig.java
```

3

The database gives back the role and all its permission rows in one query

The lookup is not a simple "find user by email". `UserRepository.findActiveByEmailWithRole` uses `JOIN FETCH u.role r JOIN FETCH r.permissions`. That means: bring the user, his role, and every permission row attached to that role, in a single trip to the SQL database. The tables involved were created in migration V1: users, roles, permissions, and the link table `role_permissions`. This is the heart of the design: a permission is a row in a table, not a line of Java code.

```
FILES backend/src/main/java/com/pms/user/repository/UserRepository.java ·
      backend/src/main/resources/db/migration/V1__schema_auth.sql ·
      backend/src/main/resources/db/migration/V2__seed_rbac.sql
```

4

Two tokens are created and signed

`JwtService` builds two JSON Web Tokens. The access token has a short life (900 seconds = 15 minutes, set in `application.yml`). It has inside it the user's email, his role name, the full list of his permission codes, his `tokenVersion` and a `firstLogin` flag. The refresh token has a long life (604800 seconds = 7 days) and it has almost nothing inside: just the email, the `tokenVersion`, and `type=refresh`. Both are signed with a secret key using HMAC-SHA. Signing means the server can see later if anything was changed, because a token that was changed no longer matches its signature.

```
FILES backend/src/main/java/com/pms/auth/service/JwtService.java ·
      backend/src/main/resources/application.yml
```

5

The two tokens come back by two different roads

`AuthController` puts the access token inside the normal JSON answer, together with the user's name, role and permission list. So the Angular app can draw the right menu at once. The refresh token is NOT in the JSON. It is written into a cookie named `pms_refresh`, with three settings. `HttpOnly` means JavaScript cannot read it, so a script pushed into the page cannot steal it. `SameSite=Strict` means the browser refuses to send it from another website. `Path=/api/auth/refresh` means the browser attaches it only to that one URL. Angular keeps the access token and the user context in `localStorage`. If `firstLogin` is true, the user is sent straight to `/change-password`.

```
FILES backend/src/main/java/com/pms/auth/controller/AuthController.java · frontend/pms-
      frontend/src/app/core/services/auth.service.ts · frontend/pms-
      frontend/src/app/features/auth/login/login.component.ts
```

6

Every later request carries the badge, and a filter reads it

Angular's `authInterceptor` sits in front of every outgoing HTTP call and adds the header `Authorization: Bearer <access token>`. On the server, `JwtAuthenticationFilter` runs before anything else. It checks the signature. It refuses tokens whose type is not "access". It reads the user's permissions again and turns each permission code into a Spring "authority". It also compares the `tokenVersion` written in the token with the `tokenVersion` column in the users table. If the two are different, the token is ignored without a word. That single whole number is how logout, password change and refresh rotation kill old tokens, even if a JWT by itself cannot be taken back. Because all this happens on every request, the server keeps no session in memory at all. `SecurityConfig` declares `SessionCreationPolicy.STATELESS`.

FILES `frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts` ·
`backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java` ·
`backend/src/main/java/com/pms/shared/config/SecurityConfig.java`

7

When the 15 minutes are over, the voucher is exchanged for a new badge

After 15 minutes the access token dies and the server answers 401. The Angular interceptor catches that 401 and calls `/api/auth/refresh`. The browser attaches the `pms_refresh` cookie by itself. The app gets a brand-new access token and sends the failed request again, quietly. The user sees nothing. Two safety rules apply. On the front end, only one refresh call can run at a time (single-flight, done with a `BehaviorSubject`), so ten 401s at the same moment do not start ten refreshes. On the back end, the refresh token is rotated, which means each refresh token can be used exactly one time. If an old one comes back, the server writes a log about a possible theft and answers 401.

FILES `frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts` · `frontend/pms-frontend/src/app/core/services/auth.service.ts` ·
`backend/src/main/java/com/pms/auth/service/AuthService.java`

8

Two separate gates decide what comes back: first the permission, then the scope

First gate. On almost every service method there is an annotation like `@PreAuthorize("hasAuthority('VIEW_PROJECT')")`. Spring runs the check before the method body and throws 403 if the authority is missing. There are 110 such annotations in the backend. Second gate. Even with `VIEW_PROJECT`, `ProjectScopeService` decides WHICH projects. If the user holds the capability `VIEW_ALL_PROJECTS`, the method returns at once and puts no filter. If not, the list is cut down to the ids given back by `findAccessibleProjectIdsByEmail`. That is a SQL query which returns the projects where the user is the `chefProject` or an active team member. `ProjectScopeInterceptor` applies the same check by itself to every URL that matches `/api/projects/{id}/**`. So no sub-resource (milestones, risks, sprints) can be reached from the side.

FILES `backend/src/main/java/com/pms/project/service/ProjectService.java` ·
`backend/src/main/java/com/pms/project/service/ProjectScopeService.java` ·
`backend/src/main/java/com/pms/project/repository/ProjectRepository.java` ·
`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java`

The real code

backend/src/main/java/com/pms/auth/service/AuthService.java

JAVA

```
50     public TokenBundle login(LoginRequest request) {
51         if (loginAttemptTracker.isBlocked(request.email())) {
52             throw new TooManyRequestsException(
53                 "Trop de tentatives de connexion. Réessayiez dans " +
54                 (LoginAttemptTracker.WINDOW_SECONDS / 60) + " minutes.");
55         }
56
57         // Look up the user – may return empty
58         User user = userRepository.findActiveByEmailWithRole(request.email()).orElse(null);
59
60         // Always run bcrypt to equalize timing (prevents user enumeration)
61         String hashToVerify = (user != null) ? user.getPasswordHash() : dummyHash;
62         boolean passwordMatches = passwordEncoder.matches(request.password(), hashToVerify);
63
64         if (user == null || !passwordMatches) {
65             loginAttemptTracker.recordFailure(request.email());
66             throw new BadCredentialsException("Identifiants incorrects");
67         }
68     }
```

What this does. The first if-block is the rate limit. If this email already failed 5 times inside a window of 15 minutes, the method stops right here and the user gets HTTP 429, so nobody can try millions of passwords. The next line loads the user from PostgreSQL, and it can give back null when the email does not exist. The line with hashToVerify is the interesting decision. When the user is null, the code compares the typed password against dummyHash on purpose. dummyHash is a fake hash computed one time at startup. bcrypt is the slow one-way scrambling function used to store passwords, and it takes the same time in both cases. So, seen from outside, an unknown email and a wrong password look the same. An attacker cannot find out which emails exist by timing the answers. The last if-block treats both failures in the same way and returns the same French message "Identifiants incorrects", again so that it does not say which half was wrong.

```
38     public String generateAccessToken(User user) {
39         List<String> permissions = user.getRole().getPermissions().stream()
40             .map(Permission::getCode)
41             .toList();
42
43         return Jwts.builder()
44             .subject(user.getEmail())
45             .claim("role", user.getRole().getName())
46             .claim("permissions", permissions)
47             .claim("tokenVersion", user.getTokenVersion())
48             .claim("type", "access")
49             .claim("firstLogin", user.isFirstLogin())
50             .issuedAt(Date.from(Instant.now()))
51             .expiration(Date.from(Instant.now().plusSeconds(accessTokenExpirySeconds)))
52             .signWith(signingKey)
53             .compact();
54     }
```

What this does. The first three lines read the rows of the `role_permissions` table through the user's role and keep only the text codes. That gives a list like `["VIEW_PROJECT", "VIEW_KPI"]`. Everything after `Jwts.builder()` fills the token. `subject` is the email, which is the identity. Each claim is one named piece of information kept inside the token: the role name, the permission list, the `tokenVersion`, `type="access"` to tell it apart from a refresh token, and `firstLogin` so the server knows the user has not chosen his own password yet. `expiration` writes a death date 900 seconds (15 minutes) later, taken from `application.yml`. `signWith(signingKey)` adds a signature computed from the secret key. Anyone can still read the content, but if you change one character the signature breaks. So a user cannot add a permission to his own token.

```
67         if (auth == null) {
68             User user = userRepository.findActiveByEmailWithRole(email).orElse(null);
69             if (user == null || user.getTokenVersion() != tokenVersion) {
70                 return;
71             }
72
73             List<SimpleGrantedAuthority> authorities = user.getRole().getPermissions().stream()
74                 .map(p -> new SimpleGrantedAuthority(p.getCode()))
75                 .toList();
76
77             auth = new UsernamePasswordAuthenticationToken(email, null, authorities);
78
79             if (cache != null) {
80                 cache.put(cacheKey, auth);
81             }
82         }
83
84         SecurityContextHolder.getContext().setAuthentication(auth);
```

What this does. This runs on every single API call. The outer if means: we did not find this user in the cache, so build him again from the database. The second if is the revocation check. If the tokenVersion written inside the token is not the same as the token_version column of the user row, the method returns and nobody is authenticated. The request is then treated as anonymous. This is how logout and password change kill, at once, a token that has not died yet. The stream turns each permission code row into a SimpleGrantedAuthority, which is only Spring's word for "a label the user carries". Those labels are exactly the strings that @PreAuthorize will test later. The result is put in a cache whose key is email + ":" + tokenVersion, so an old revoked token can never reuse the cache entry of a new session. The last line puts the rebuilt identity into the SecurityContextHolder, a holder that lives for one request and that the rest of the application reads.

```
99         int presented = jwtService.extractTokenVersion(refreshTokenValue);
100         if (presented != user.getTokenVersion()) {
101             // Stale version → token was already rotated or explicitly revoked.
102             // Could indicate refresh-token theft – log prominently.
103             log.warn("Refresh token replay detected for {} (presented={}, current={}). Possible
104 token theft.",
105                     email, presented, user.getTokenVersion());
106             throw new JwtException("Token révoqué");
107         }
108
109         // Rotation: bump tokenVersion so the current refresh token can never be reused
110         int oldVersion = user.getTokenVersion();
111         user.revokeAllTokens();
112         User saved = userRepository.save(user);
```

What this does. This is the core rule of the refresh endpoint. It reads the version number carried by the refresh token and compares it with the version stored right now on the user row. If they are different, the token was already used, or the user logged out. So the server refuses with 401 and writes a warning, because an old refresh token that comes back is a believable sign that somebody stole it. If they are the same, the token is accepted, and it is burned at once. `revokeAllTokens()` only does `tokenVersion++` on the user row. That makes the refresh token that was just used invalid for any future call. This is called rotation, and it means each refresh token works exactly one time. The saved user, who now has the new version, is then used to make the next pair of tokens.

```
36     /** Vrai si l'utilisateur courant détient la capacité d'accès global (pas de filtre de
37     périmètre). */
38     public boolean hasAllAccess() {
39         Authentication auth = SecurityContextHolder.getContext().getAuthentication();
40         if (auth == null) return false;
41         return auth.getAuthorities().stream()
42             .map(GrantedAuthority::getAuthority)
43             .anyMatch(ALL_ACCESS::equals);
44     }
45
46     /** Identifiants des projets accessibles à l'utilisateur (chef de projet U affectations
47     actives). */
48     @Transactional(readOnly = true)
49     public Set<Long> accessibleProjectIds(String email) {
50         if (email == null) return Set.of();
51         return projectRepository.findAccessibleProjectIdsByEmail(email);
52     }
53
54     /** Lève {@link AccessDeniedException} (→ 403) si le projet est hors périmètre de
55     l'utilisateur. */
56     @Transactional(readOnly = true)
57     public void assertCanAccess(Long projectId, String email) {
58         if (hasAllAccess()) return;
59         if (!accessibleProjectIds(email).contains(projectId)) {
60             throw new AccessDeniedException("Projet hors périmètre");
61         }
62     }
63 }
```

What this does. `hasAllAccess()` asks one question only: does the current user carry the label `VIEW_ALL_PROJECTS` (the constant `ALL_ACCESS`)? Notice that it never tests a role name such as `"DIRECTEUR"`. The rule is written against a capability, so an administrator can give the same power to a new role tomorrow without touching Java. `accessibleProjectIds()` asks the database for the ids of the projects the user is personally attached to. The SQL behind it (`ProjectRepository.findAccessibleProjectIdsByEmail`) returns the projects where the user is the `chefProjet` OR has an active row in `team_assignments`, and it leaves out deleted and archived projects. `assertCanAccess()` puts the two together. If the user has the global capability, it returns straight away with no filter. If not, and the project id he asks for is not in his personal set, it throws `AccessDeniedException`. The global exception handler turns that into HTTP 403 "Accès refusé".

```

195     readonly visibleSections = computed(() => {
196         const perms = this.auth.permissions();
197         return this.NAV
198             .map(s => ({ ...s, items: s.items.filter(i => !i.permission ||
199 perms.includes(i.permission)) })))
200             .filter(s => s.items.length > 0);
    });

```

What this does. This is the whole reason why the same page shows a different menu to different users. NAV is a fixed list of menu entries. Each entry can carry a permission string such as 'MANAGE_ROLES' or 'VIEW_BILLING'. perms is the array of permission codes the server sent back at login. The .map keeps, inside each section, only the entries whose permission is there, or the entries that have no permission at all, like the Dashboard. The .filter then drops a whole section heading once that section has become empty. So a Développeur never sees an "Administration" title with nothing under it. computed() is Angular's reactive wrapper: if the permission list changes, the menu draws itself again. There is not one single if (role === 'DIRECTEUR') anywhere. The menu is just a picture of database rows.

Extra explanations

THE AUTH SECTION GIVES THE READER THE IDEA THAT "A DIRECTEUR SEES EVERYTHING, A CHEF DE PROJET SEES ONLY HIS PROJECTS". FOR MONEY, THE PERMISSION DATA SAYS THE OPPOSITE. V12__FIX_RBAC_MATRIX.SQL GIVES MANAGE_BILLING TO CHEF_PROJET AND ONLY VIEW_BILLING TO DIRECTEUR. A BEGINNER WHO READS THE TWO SECTIONS WILL NOT BE ABLE TO EXPLAIN WHY THE BOSS CANNOT ADD A MILESTONE.

Scope and power are two different things, and in PMS they point in opposite directions for billing on purpose. A Directeur has the widest scope. He holds VIEW_ALL_PROJECTS, so ProjectScopeService.hasAllAccess() returns true and he reads every project in the company. But scope answers "on which rows", not "what am I allowed to do". The permission grid in backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql gives DIRECTEUR only VIEW_BILLING. CHEF_PROJET gets both MANAGE_BILLING and VIEW_BILLING, with the comment "vue seule : facturation = PM (BR-038)". So a Directeur can read every billing plan in the firm, but he cannot create a milestone on any of them. And a Chef de Projet can create milestones, but only on the few projects where he is the chefProjet or an active member of the team. In short: the Directeur is wide and read-only, and the Chef de Projet is narrow and hands-on. That file also shows how such a change is made. V12 first DELETES every grant of the four business roles, then puts the target grid back in. The behaviour changes and not one line of Java is touched.

PROJECTSCOPEINTERCEPTOR IS NAMED IN SECTION 2, BUT THE GUIDE NEVER SAYS WHERE IT IS PLUGGED IN. IT NEVER SAYS WHEN IT RUNS COMPARED WITH @PREAUTHORIZE, AND IT NEVER SAYS WHY A BILLING URL IS PROTECTED BY A CLASS THAT LIVES IN THE PROJECT MODULE.

Every URL that looks like `/api/projects/{id}/...` is guarded on its own, and the guard is not in the billing code. `backend/src/main/java/com/pms/shared/config/WebMvcConfig.java` registers an interceptor with `registry.addInterceptor(projectScopeInterceptor).addPathPatterns("/api/projects/**")`. An interceptor is a class that Spring runs between the security filter and the controller. The interceptor itself is `backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java`. It matches the URL against the pattern `^/api/projects/(\d+)/(.*)?$`, takes the project number out of the address, and calls `scopeService.assertCanAccess(projectId, auth.getName())`. That call throws `AccessDeniedException` (HTTP 403) when the project is outside your perimeter. The order matters. The JWT filter says who you are. Then this interceptor checks the project. Then the controller runs. Then `@PreAuthorize` on the service checks what you are allowed to do. The payoff is that milestones, payments, amendments, risks, deliverables and sprints all get project-scope protection for free, just by living under `/api/projects/{id}/`. And no developer can forget to write the check in a new module.

SECURITYCONFIG TURNS CSRF PROTECTION OFF (.CSRF(ABSTRACTHTTPCONFIGURER::DISABLE)) WHILE THE REFRESH TOKEN LIVES IN A COOKIE. THE GUIDE PRAISES SAMESITE=STRICT BUT NEVER SAYS THAT CSRF IS OFF, AND NEVER SAYS WHY THAT PAIR IS DEFENSIBLE. THIS IS A CLASSIC JURY QUESTION ON A PROJECT WITH A SECURITY FLAVOUR.

One line in `backend/src/main/java/com/pms/shared/config/SecurityConfig.java` reads `.csrf(AbstractHttpConfigurer::disable)`. CSRF (Cross-Site Request Forgery) is the attack where another website makes your browser send a request to PMS and quietly reuses your cookie. The usual defence is a CSRF token, and PMS turns it off on purpose, for two reasons. First, the thing that authorises a business call is not a cookie. It is the `Authorization: Bearer` header, which the Angular interceptor adds by hand. Another website cannot make your browser add that header, so a forged request arrives with no proof of who you are and is refused. Second, the one cookie that does exist, `pms_refresh`, is declared `SameSite=Strict`. The browser then refuses to send it at all when the request starts from another site. And `Path=/api/auth/refresh` means it is never attached to a business endpoint anyway. Here is the honest way to say this at the defence. CSRF protection is off because the session is stateless and the cookie is both strictly same-site and limited to one path. If PMS ever used a cookie to authenticate a business endpoint, the token would have to come back.

Words used in this chapter

JWT (JSON Web Token)	A small piece of text, made of three parts separated by dots, that holds readable facts about the user plus a signature which proves the server wrote it and nobody changed it.
Access token	The short-lived JWT (15 minutes in PMS) that goes with every API call in the <code>Authorization</code> header and that carries the user's permission list.

Refresh token	The long-lived JWT (7 days in PMS) kept in an HttpOnly cookie and used only to get a new access token, never to call a business endpoint.
Stateless	The server keeps nothing about your session between two requests, so it has to work out again who you are from the token you show each time (SessionCreationPolicy.STATELESS in SecurityConfig).
bcrypt hash	A slow-on-purpose one-way change of a password, kept in the password_hash column, from which you cannot get the first password back.
Permission code	A short uppercase string such as VIEW_ALL_PROJECTS that lives as a row in the permissions table and is tied to roles through the role_permissions table.
Authority (Spring Security)	Spring's own name for one label stuck on the user who is logged in; in PMS each authority is exactly one permission code.
@PreAuthorize	A Java annotation put above a service method that makes Spring test a condition, such as hasAuthority('VIEW_PROJECT'), and refuse the call with HTTP 403 before the method body runs.
Scope (périmètre)	The extra filter that decides which rows of data a user may see, on top of the question whether he may use the feature at all.
tokenVersion	A whole number kept both in the users table and inside every token; adding one to the column makes every token given to that user before stop working at once.

Traps to avoid

You might think a JWT is encrypted and so it is secret, but actually it is only signed. Anybody who copies the token can decode it and read your email, your role and your full permission list. What the signature stops is a change: you cannot add `MANAGE_USERS` to your own token, because the server would compute the signature again and refuse it.

You might think the Angular permissionGuard and the sidebar filter protect the data, but actually they only hide links and buttons. Anybody can call the API straight away with a tool like curl. The real protection is `@PreAuthorize` on the backend service plus `ProjectScopeService`. The frontend checks are there only so that the user is not shown doors that would shut in his face.

You might think that holding `VIEW_PROJECT` means you see every project, but actually permission and scope are two separate gates. A Chef de Projet and a Directeur both hold `VIEW_PROJECT`. The difference is that the Directeur's role also holds `VIEW_ALL_PROJECTS` (added by migration V13). So `ProjectScopeService.hasAllAccess()` returns true and skips the filter, while the Chef de Projet gets only the ids given back by `findAccessibleProjectIdsByEmail` — the projects where he is chefProjet or an active team member.

You might think logging out destroys the token, but actually a JWT cannot be taken back once it has been given out; in pure maths it stays valid until its expiry date. PMS solves this by adding one to the `token_version` column in the users table. `JwtAuthenticationFilter` compares the version inside the token with the version in the database on every request, so all the old tokens stop being accepted at once.

You might think that changing what a role can do needs you to edit Java code and deploy again, but actually permissions are rows in `role_permissions`. When an administrator ticks a box in the Roles admin page, it calls `RoleAdminService.update()`. That saves the new rows and then clears the `securityContext` cache. So users who are already connected build their authorities again from the database on their very next request.

The financial engine (KPI calculation, milestone billing and budget revisions)

It takes the days people work, and what each of those days costs the company, and turns them into live money figures: what the project has already spent, what it will cost in total by the end, and how much of the price sold to the client is left over as profit.

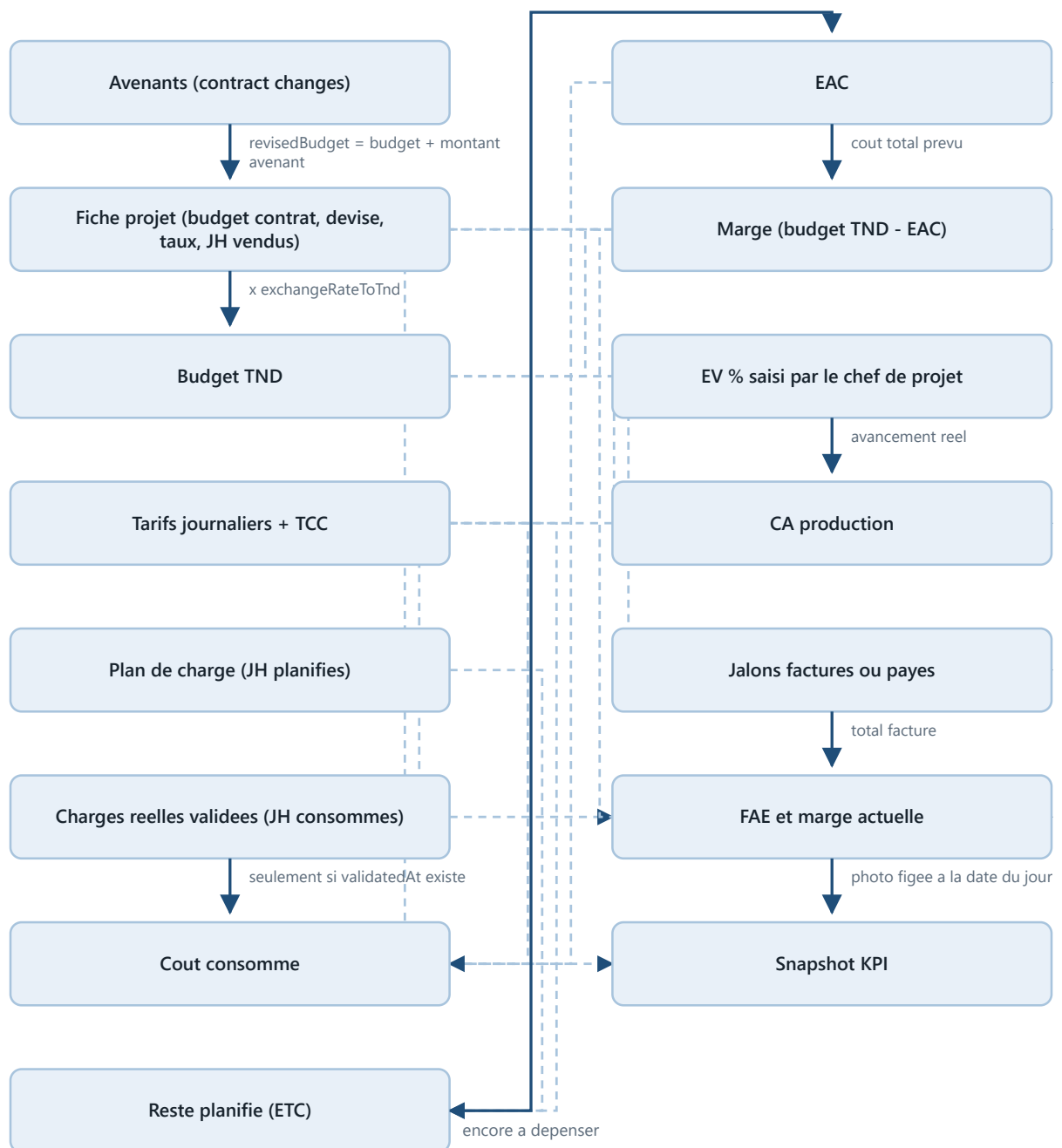
THINK OF IT LIKE...

Think of a caterer who agrees on a fixed price of 100 000 for a wedding dinner. The agreed price is the budget. It never moves, unless the couple orders extra dishes — that is an "avenant", a signed change to the contract. Every hour the cooks work costs the caterer real money, so the cost grows while they work. The profit (the margin) is just the agreed price minus what the whole dinner will have cost when the last plate leaves the kitchen. The caterer also invoices the couple in slices - 30% when the menu is signed, 40% on the day, 30% after. Those slices are billing milestones. They are about cash coming in, not about profit.

WHY THIS EXISTS

Before PMS, ST2i steered its projects with one Excel workbook per project (docs/DEV_Fiche_revue_projet-Real.xlsx). Each workbook held the identification sheet, the internal quote, the workload plan and the monthly review, with formulas typed by hand to link them. That way of working breaks down when the company grows. A formula can be overwritten by mistake. Two project managers can compute the margin in two different ways. Nobody can see the whole portfolio on one screen. There is no history of what the numbers looked like last month. And everyone who can open the file can read the cost rates of everyone else, which come from their salaries. The financial engine writes those Excel formulas again, one time, in one place (backend/src/main/java/com/pms/kpi/service/KpiService.java). It applies them in the same way to every project. It computes them again from the raw data on every request, so they can never be out of date. And it hides them behind a permission (VIEW_KPI), so a developer sees his own workload but not the money.

The picture



From days and rates to the financial indicators — Notice that the workload plan and the real declarations both feed the same cost formula. Notice too that the plan only stays alive for the months where there is no validated real work. That single filter is what makes EAC an estimate, instead of counting the same thing twice.

Step by step

1

The project sheet holds the numbers from the contract

A project row stores what was signed with the client: the initial budget, the currency (a Tunisian project is in TND, a West-African one may be in FCFA), the exchange rate to TND, and the workload sold in man-days (a man-day, "jour-homme" or JH, is one person working for one day). The budget is the price the client agreed to pay. It is not an allowance you are free to spend. Two helper methods make the first two decisions. `getEffectiveBudget()` takes the revised budget when a contract change exists, and the initial one when there is none. `getBudgetTnd()` multiplies it by the exchange rate, so that every later figure is in Tunisian dinars. A third helper, `getPprTnd()`, gives back 5% of that budget: the provision pour risques, a safety cushion the company keeps in mind in case something goes wrong. It is shown on the project sheet, and the KPI engine never takes it off anywhere.

```
FILES backend/src/main/java/com/pms/project/entity/Project.java ·  
      backend/src/main/java/com/pms/project/dto/ProjectResponse.java
```

2

Two tables of days: what was planned, and what really happened

A `PlanCharge` row says "this person is planned for this many days in this month" (one row per person per month). A `ChargeReelle` row says "this person really worked this many days in this month", and it only counts once a manager has validated it — the repository query keeps only the rows where `validatedAt` IS NOT NULL. This is why a developer who types his own days cannot make the margin look better: a declaration that is not validated is invisible to the engine. In this project, a "charge" always means workload counted in man-days, never money.

```
FILES backend/src/main/java/com/pms/workload/entity/PlanCharge.java ·  
      backend/src/main/java/com/pms/workload/entity/ChargeReelle.java ·  
      backend/src/main/java/com/pms/workload/repository/ChargeReelleRepository.java
```

3

Each day is turned into money

Days are not money, so the engine multiplies each day by a rate. Every person has a `Resource` row with a `dailyRate` (the raw cost of one day of that person) and a `tccRate`. The `tccRate` is a percentage of extra company charges added on top of the salary cost - social contributions, office, management. So the cost of one line is $\text{days} \times \text{dailyRate} \times (1 + \text{tccRate})$. A person is not paid the same in 2024 and in 2025, so a `TccAnnuel` row can replace both numbers for a given year, and the engine takes the rate of the year in which the days were worked.

```
FILES backend/src/main/java/com/pms/user/entity/Resource.java ·  
      backend/src/main/java/com/pms/user/entity/TccAnnuel.java ·  
      backend/src/main/java/com/pms/kpi/service/KpiService.java
```

4

EAC: what the project will have cost when it is finished

EAC means Estimate At Completion. It is today's answer to the question "how much will this project have cost me in total, in the end?". The engine computes it as the money already spent on validated days, plus the money still planned for the months that have not produced validated days yet. The trick is in the key user-id + ":" + period: as soon as a person has a validated real line for a month, the planned line of that same person and that same month is dropped. So the same month is never counted twice. The margin is then just the budget in TND minus EAC. And `tauxConsommation` (consumption rate) is the money already spent divided by the budget, a number between 0 and 1.

FILES `backend/src/main/java/com/pms/kpi/service/KpiService.java`

5

The same story told in days instead of money

Three indicators tell the same story in man-days, because a project manager thinks in days. `consommeJh` is the sum of the validated days. `rafJh` is the "reste a faire" (the work left to do), taken from the planned lines that are still there. `deriveJh` is the drift: the days sold to the client, minus the days used, minus the days left. A drift of zero means the team will finish with exactly the workload that was sold. A negative drift means the team will need more days than the client paid for, and that is the early warning sign the Excel file never gave clearly.

FILES `backend/src/main/java/com/pms/kpi/service/KpiService.java`

6

The number a human types in: EV %, and the figures on the revenue side

Earned Value (EV) is the share of the contract that has really been delivered, written as a percentage. The engine does not guess it. At the monthly review the project manager types it into the snapshot form (`SnapshotRequest.evPct`, which must be between 0 and 100), and the live screen takes the value from the most recent snapshot. Three figures come out of it. CA production = budget TND x EV / 100, which is the revenue the company is allowed to count as earned. total facture is the sum of the milestones already invoiced or paid. And FAE (facture a etablr, "invoice still to be issued") is the earned revenue minus the invoiced revenue - work already done that the client has not been billed for yet. The current margin is CA production minus the cost really spent. If nobody has ever entered an EV, the engine gives back null for these and pushes a warning that explains why, instead of a fake zero.

FILES `backend/src/main/java/com/pms/kpi/dto/SnapshotRequest.java` ·
`backend/src/main/java/com/pms/kpi/service/KpiService.java`

7

Worked example, part 1: the cost side

Take a project with a budget of 100 000 TND. The currency is TND, so the exchange rate is 1, and 100 man-days were sold. One profile works on it: dailyRate 500 TND and tccRate 0.20, so one day costs $500 \times 1.20 = 600$ TND. The plan de charge holds 10 monthly lines of 10 days each = 100 planned days. Four months are over and validated, 10 days each, so 40 days are used. $\text{budgetConsume} = 40 \times 600 = 24\,000$ TND. The six months with no validated real days are still planned: $\text{etcCost} = 60 \times 600 = 36\,000$ TND. $\text{EAC} = 24\,000 + 36\,000 = 60\,000$ TND. $\text{marge} = 100\,000 - 60\,000 = 40\,000$ TND, which is 40% of the contract. $\text{tauxConsommation} = 24\,000 / 100\,000 = 0.2400$, so 24% of the budget is burnt. In days: $\text{consommeJh} 40$, $\text{rafJh} 60$, $\text{deriveJh} = 100 - 40 - 60 = 0$, so the plan sits exactly on the workload that was sold. Now change one thing. The team really burnt 12.5 days per month, so 50 days instead of 40. budgetConsume becomes $50 \times 600 = 30\,000$. EAC becomes $30\,000 + 36\,000 = 66\,000$. The margin falls from 40 000 to 34 000. And deriveJh becomes $100 - 50 - 60 = -10$, ten days more than were sold. That one negative number is the whole point of the engine.

FILES backend/src/main/java/com/pms/kpi/service/KpiService.java ·
 backend/src/main/java/com/pms/user/entity/Resource.java

8

Worked example, part 2: billing, the revenue side, and the snapshot

Same project, back to the healthy case (24 000 spent). A billing milestone ("jalón") is created at 30% and invoiced. computeMontant gives $100\,000 \times 30 / 100 = 30\,000$ TND, and because its status is FACTURE it counts, so $\text{totalFacture} = 30\,000$. At the review the project manager judges the delivered work at 45%, so CA production = $100\,000 \times 45 / 100 = 45\,000$ TND. $\text{FAE} = 45\,000 - 30\,000 = 15\,000$ TND already earned but not yet invoiced. $\text{margeActuelle} = 45\,000 - 24\,000 = 21\,000$ TND, and $\text{margeActuellePct} = 21\,000 / 45\,000 = 0.4667$, so a margin of 46.67% on the work done so far. The PPR is $100\,000 \times 0.05 = 5\,000$ TND, shown on the project sheet as the risk cushion. If the client then signs an avenant of +20 000, the revised budget becomes 120 000. The milestones still in PREVU are priced again by the system (a 30% milestone would become 36 000), while the 30 000 already invoiced stays frozen. Pressing "snapshot" writes all of these figures into one snapshot_kpis row dated today - a photograph of the project on that day, one per day at most. The live screen, on the other hand, computes everything again from the raw days and rates on every single GET.

FILES backend/src/main/java/com/pms/billing/service/JalonService.java ·
 backend/src/main/java/com/pms/billing/service/AvenantService.java ·
 backend/src/main/java/com/pms/kpi/entity/SnapshotKpi.java ·
 backend/src/main/java/com/pms/kpi/controller/KpiController.java

The real code

backend/src/main/java/com/pms/project/entity/Project.java

JAVA

```
100     public BigDecimal getEffectiveBudget() {
101         return revisedBudget != null ? revisedBudget : initialBudget;
102     }
103
104     /** Durée du contrat en jours (bornes incluses, comme la Fiche Excel). */
105     public Long getDurationDays() {
106         if (startDate == null || endDate == null) return null;
107         return ChronoUnit.DAYS.between(startDate, endDate) + 1;
108     }
109
110     /** Budget effectif converti en TND (devise pragmatique : budget × taux). */
111     public BigDecimal getBudgetTnd() {
112         BigDecimal eff = getEffectiveBudget();
113         if (eff == null) return null;
114         BigDecimal rate = exchangeRateToTnd != null ? exchangeRateToTnd : BigDecimal.ONE;
115         return eff.multiply(rate);
116     }
117
118     /** Provision Pour Risques = budget TND × 5 % (Fiche identification). */
119     public BigDecimal getPprTnd() {
120         BigDecimal tnd = getBudgetTnd();
121         return tnd != null ? tnd.multiply(new BigDecimal("0.05")) : null;
122     }
```

What this does. These three methods are computed on the spot. Nothing is stored in the database for them: they are computed again every time somebody asks. `getEffectiveBudget()` decides which budget is the real one. If a revised budget exists, because a change to the contract was signed, it wins. If not, the initial budget is used. So the rest of the engine never has to ask that question again. `getDurationDays()` counts how long the contract lasts, with the first day and the last day both counted, exactly like the Excel sheet did, and that is why there is a "+ 1". `getBudgetTnd()` multiplies the effective budget by the exchange rate. Costs are always computed from daily rates written in Tunisian dinars, so the budget must be pushed into the same unit before anything is taken away from it. `getPprTnd()` is the provision pour risques: a flat 5% of the budget, the cushion the company keeps in reserve for accidents. Note that it is only shown on screen; the KPI engine never takes it off the margin.

```
262     /** Coût d'une charge : JH x tarif TCC de l'année de la période (F-AFF-13 §6.3 règle 4),
263     sinon tarif de base. */
264     private BigDecimal chargeCost(BigDecimal days, Long userId, LocalDate period,
265                                   Map<Long, Resource> resourceMap,
266                                   Map<Long, Map<Integer, TccAnnuel>> tccByUserYear) {
267         TccAnnuel yearly = period != null
268             ? tccByUserYear.getOrDefault(userId, Map.of()).get(period.getYear())
269             : null;
270         if (yearly != null) {
271             return days.multiply(yearly.getDailyRate())
272                 .multiply(yearly.getTccRate().add(BigDecimal.ONE))
273                 .setScale(2, RoundingMode.HALF_UP);
274         }
275         Resource resource = resourceMap.get(userId);
276         if (resource == null) return BigDecimal.ZERO;
277         return days.multiply(resource.getDailyRate())
278             .multiply(resource.getTccRate().add(BigDecimal.ONE))
279             .setScale(2, RoundingMode.HALF_UP);
280     }
```

What this does. This tiny method is where days become money, and every single money figure in the engine goes through it. First it looks for a rate that belongs to the year the work was done (`period.getYear()`), because a person does not cost the same in 2024 and in 2025; if such a yearly rate exists, it wins. The formula is always the same: $\text{days} \times \text{dailyRate} \times (1 + \text{tccRate})$. The `tccRate` is the extra company charge written as a fraction, so 0.20 means twenty percent on top - that is why the code writes `getTccRate().add(BigDecimal.ONE)` instead of a plain multiplication. If there is no yearly rate, it falls back to the person's base `Resource` rate. The last decision is on purpose, and it is a little dangerous. If a person has no `Resource` at all, the method gives back zero instead of crashing, and the caller makes up for it by adding a warning you can see ("Tarif journalier manquant... coût compté à 0"), so the reader knows the margin is too optimistic. Everything is `BigDecimal` with two decimals and `HALF_UP` rounding, because money must never be kept in a floating-point type.

```

177         // EAC : coût réel + coût planifié des périodes sans charge réelle validée
178         Set<String> actualKeys = actualCharges.stream()
179             .map(cr -> cr.getUser().getId() + ":" + cr.getPeriod())
180             .collect(Collectors.toSet());
181
182         List<PlanCharge> remainingPlan = planCharges.stream()
183             .filter(pc -> !actualKeys.contains(pc.getUser().getId() + ":" + pc.getPeriod()))
184             .toList();
185
186         BigDecimal etcCost = remainingPlan.stream()
187             .map(pc -> chargeCost(pc.getPlannedDays(), pc.getUser().getId(), pc.getPeriod(),
188 resourceMap, tccByUserYear))
189             .reduce(BigDecimal.ZERO, BigDecimal::add);
190
191         BigDecimal eac = budgetConsome.add(etcCost);
192
193         // Budget converti en TND (les coûts sont calculés via des tarifs journaliers en TND) –
194 ADR-020 :
195         // toutes les grandeurs KPI sont ainsi homogènes en TND, quelle que soit la devise du
196 projet.
197         BigDecimal budgetTnd = project.getBudgetTnd();
198         BigDecimal marge = budgetTnd != null ? budgetTnd.subtract(eac) : null;
199         BigDecimal tauxConsommation = null;
200         if (budgetTnd != null && budgetTnd.compareTo(BigDecimal.ZERO) > 0) {
201             tauxConsommation = budgetConsome.divide(budgetTnd, 4, RoundingMode.HALF_UP);
202         }

```

What this does. This block is the heart of the engine. It builds a set of text keys shaped like "userId:month", one for every validated real line. Then it keeps only the planned lines whose key is not in that set. In plain words: a month that has already produced validated work is not a forecast any more, so its plan is thrown away and only the real figure counts. The cost of the planned lines that are left is the ETC (Estimate To Complete, what is still to be spent). And EAC = money already spent + ETC, which is the total cost the project is heading for. The margin is then the budget in TND minus EAC, so it is a forecast of the final profit, not a picture of today. Two guards matter here. The margin stays null when the project has no budget yet, instead of pretending that the margin is minus the cost. And the division for `tauxConsommation` is done only when the budget is strictly above zero, because dividing by zero would throw. The comment points to ADR-020: everything is converted to TND first, so that taking a cost away from a budget is always valid, even for a project sold in FCFA.

```
203         // Charge totale consommée / RAF / Dérive (en JH, contre le workload VENDU – pas le plan
204         de charge)
205         BigDecimal consommeJh = actualCharges.stream()
206             .map(ChargeReelle::getActualDays).reduce(BigDecimal.ZERO, BigDecimal::add);
207         BigDecimal rafJh = remainingPlan.stream()
208             .map(PlanCharge::getPlannedDays).reduce(BigDecimal.ZERO, BigDecimal::add);
209         BigDecimal deriveJh = project.getSoldWorkloadDays() != null
210             ? project.getSoldWorkloadDays().subtract(consommeJh).subtract(rafJh)
211             : null;
```

What this does. Here the same reasoning is done again in man-days instead of dinars, because that is the language a project manager really speaks. `consommeJh` adds up the validated days worked. `rafJh` (reste à faire, the work left to do) adds up the planned days of the months that have not happened yet. Notice that it uses `remainingPlan`, the very same filtered list used for the EAC, so days and money can never tell two different stories. `deriveJh` is the drift, and the comment insists on the important decision: it is measured against the workload sold to the client, not against the internal plan. A positive drift means the team expects to finish under the days sold. Zero means exactly on target. And a negative number means the project will use more days than the client is paying for.

```

225         BigDecimal caProduction = null;
226         if (evPct != null && budgetTnd != null) {
227             caProduction = budgetTnd.multiply(evPct).divide(HUNDRED, 2, RoundingMode.HALF_UP);
228         } else if (evPct == null) {
229             warnings.add("Avancement EV non renseigné – créer un snapshot avec l'EV % pour
230 activer CA production, FAE et marge actuelle");
231         }
232
233         BigDecimal exchangeRate = project.getExchangeRateToTnd() != null
234             ? project.getExchangeRateToTnd() : BigDecimal.ONE;
235         BigDecimal totalFacture =
236 jalonFacturationRepository.findActiveByProjectId(projectId).stream()
237             .filter(j -> j.getStatut() == JalonStatut.FACTURE || j.getStatut() ==
238 JalonStatut.PAYE)
239             .map(JalonFacturation::getMontant)
240             .filter(m -> m != null)
241             .reduce(BigDecimal.ZERO, BigDecimal::add)
242             .multiply(exchangeRate)
243             .setScale(2, RoundingMode.HALF_UP);
244
245         BigDecimal fae = caProduction != null ? caProduction.subtract(totalFacture) : null;
246
247         // Marge actuelle = CA production - coût actuel
248         BigDecimal margeActuelle = caProduction != null ? caProduction.subtract(budgetConsomme) :
249 null;
250
251         BigDecimal margeActuellePct = null;
252         if (margeActuelle != null && caProduction.compareTo(BigDecimal.ZERO) > 0) {
253             margeActuellePct = margeActuelle.divide(caProduction, 4, RoundingMode.HALF_UP);
254         }

```

What this does. This is the revenue half of the engine, and one human number drives it: `evPct`, the percentage of the contract that the project manager declares as really delivered. CA production is that percentage applied to the value of the contract, so it is the revenue the company has earned so far, even if no invoice has gone out yet. When the manager has never entered an EV, the code refuses to invent one and pushes a warning message instead. `totalFacture` adds up only the milestones whose status is `FACTURE` or `PAYE` - a milestone still in `PREVU` is a plan, not money, so it is left out - and multiplies the total by the exchange rate to stay in TND. FAE (facture a etabli) is the gap between what has been earned and what has been invoiced: a positive FAE means the company is late in billing work it has already done. Last, `margeActuelle` compares the earned revenue with the cost really spent, which answers the question "am I making money on the part I have already delivered?". Its percentage is computed only when CA production is strictly above zero, so that there is no division by zero.

```

138     private void validatePourcentageSum(Long projectId, BigDecimal oldPct, BigDecimal newPct) {
139         BigDecimal current = jalonRepository.sumPourcentageByProjectId(projectId);
140         BigDecimal effective = current.subtract(oldPct != null ? oldPct :
141         BigDecimal.ZERO).add(newPct);
142         if (effective.compareTo(BigDecimal.valueOf(100)) > 0) {
143             throw new BusinessException(
144                 "La somme des pourcentages dépasserait 100% (actuel : " +
145                 current.toString() + "%)");
146         }
147     }
148
149     private BigDecimal computeMontant(Project project, BigDecimal pourcentage) {
150         BigDecimal budget = project.getEffectiveBudget();
151         if (budget == null) return null;
152         return budget.multiply(pourcentage).divide(BigDecimal.valueOf(100), 2,
153         RoundingMode.HALF_UP);
154     }

```

What this does. A billing milestone ("jalón") is written as a share of the contract, and these two methods hold the two rules around it. `validatePourcentageSum` refuses the operation when the milestones of a project would add up to more than 100% of the contract, because that would mean invoicing the client more than he agreed to pay. The clever part is the `oldPct` argument: it takes away the milestone's own earlier share before adding the new one, so changing a 30% milestone into a 35% one is measured the right way instead of being counted twice. The error is a `BusinessRuleException`, a special exception type that the application turns into an HTTP 409 Conflict, so the user gets a message he can read instead of a stack trace. `computeMontant` then turns the percentage into a real amount: it multiplies the effective budget by the percentage and divides by 100 - so with a budget of 100 000, a 30% milestone is worth 30 000. Notice that the amount comes from `getEffectiveBudget()`. That is why a signed change to the contract automatically changes the price of the milestones that have not been invoiced yet.

Extra explanations

NOTHING IN THE GUIDE LINKS THE ERROR SHAPE OF THE BACKEND TO THE WAY THE FRONTEND SHOWS ERRORS, AND THERE IS A REAL MISMATCH THERE. THE BACKEND RETURNS A PROBLEMDetail WHOSE MESSAGE FIELD IS CALLED detail, BUT BILLING.COMPONENT.TS READS e.error?.message: ERROR: (E) => { THIS.MODALERROR.SET(E.error?.message ?? 'ERREUR. '); ... }. PROBLEMDetail HAS NO MESSAGE FIELD. SO THE CAREFUL FRENCH MESSAGE "LA SOMME DES POURCENTAGES DÉPASSERAIT 100%" NEVER REACHES THE USER, WHO ONLY SEES "ERREUR.". THE AUTHOR MUST BE ABLE TO EXPLAIN THIS AT THE DEFENCE.

When the backend says no to a milestone, the body of the answer follows RFC 7807. That is the web standard for error bodies, and Spring gives it to you with the class `ProblemDetail`. It looks like this: `{"type":"/errors/business-rule","title":"Unprocessable Entity","status":422,"detail":"La somme des pourcentages dépasserait 100% (actuel : 80%)"}.` The sentence for the human is in the field named `detail`. In `frontend/pms-frontend/src/app/features/billing/billing.component.ts` the error branch reads `e.error?.message` today. `ProblemDetail` has no field called `message`, so the expression falls back to

the default, and the modal shows the plain word "Erreur.". The rule is still applied correctly on the server, and nobody can save 130% of a budget. But the explanation is lost on the way to the screen. The fix is one word: `read e.error?.detail ?? e.error?.message ?? 'Erreur.'`.

THE GUIDE SAYS "THE CONTROLLER READS THE BODY" AND "THE SERVICE APPLIES THE RULE". BUT IT NEVER EXPLAINS THAT PMS CHECKS THE SAME NUMBER THREE SEPARATE TIMES, IN THREE DIFFERENT TECHNOLOGIES, AND WHY THAT IS DONE ON PURPOSE.

A milestone percentage is checked three times, and each check catches a different kind of mistake. First, the shape. In `backend/src/main/java/com/pms/billing/dto/JalonRequest.java` the fields have annotations on them: `@NotBlank String label`, `@NotNull @Positive @DecimalMax("100") BigDecimal pourcentage`. An annotation is a label written above a field that another program reads. Here the `@Valid` on the controller parameter tells Spring to run these checks before your own code starts, and a failure becomes HTTP 400. This catches values that make no sense, such as 0, -5 or 140. Second, the business rule. The annotation can only look at one milestone at a time, so it cannot know that this project already has 80% of milestones. For that you need the database, so this check lives in `JalonService.validatePourcentageSum`. That method adds up the percentages that already exist and says no if the new total goes over 100. Third, the safety net. `V9__schema_billing.sql` writes `CONSTRAINT chk_jf_pourcentage CHECK (pourcentage > 0 AND pourcentage <= 100)` straight into PostgreSQL. So even a script, or a future service that forgot the rule, cannot write a bad row. An easy way to remember it: the annotation checks the field, the service checks the situation, and the database says no to the impossible.

THE BILLING SCREEN SHOWS A TOTAL AT THE TOP LABELLED "TOTAL FACTURÉ :" (TOTAL INVOICED). IT IS COMPUTED AS THIS `JALONS().REDUCE((S, J) => S + J.MONTANT, 0)`, SO IT ADDS UP EVERY MILESTONE WHATEVER ITS STATUS. KPISERVICE.TOTALFACTURE ONLY ADDS UP THE MILESTONES WHOSE STATUS IS FACTURE OR PAYE. TWO FIGURES WITH THE SAME NAME CAN BE DIFFERENT ON THE SAME PROJECT, AND THE GUIDE NEVER WARNS ABOUT IT.

Be careful with the word "facturé": the billing screen and the KPI screen do not count the same thing. In `frontend/pms-frontend/src/app/features/billing/billing.component.ts` the figure at the top comes from `jalonTotal()`, which adds the montant of every milestone in the table. That includes the ones still in status `PREVU`, which have not been invoiced at all. So it is really "the total value of the billing plan". In `backend/src/main/java/com/pms/kpi/service/KpiService.java` the figure called `totalFacture` filters first, with `.filter(j -> j.getStatut() == JalonStatut.FACTURE || j.getStatut() == JalonStatut.PAYE)`. So it only counts money that was really invoiced, and that is the one used to compute FAE (earned revenue minus invoiced revenue). Take a project with a 30% milestone invoiced and a 40% milestone still planned. The billing header shows 70% of the budget and the KPI screen shows 30%. Each one is right for its own job, but the label on the billing page is misleading and should read "Total du plan de facturation".

Words used in this chapter

Charge (in JH, jour-homme / man-day)	It is an amount of work counted in days of one person's time, never a sum of money - 40 JH means someone worked forty days.
TCC (taux de charges complémentaires)	It is the extra percentage added on top of a person's raw daily rate to pay for social contributions, offices and management, so a rate of 0.20 means one day really costs 1.20 times the daily rate.
EAC (Estimate At Completion)	It is today's best guess of what the project will have cost in total once it is finished, worked out as the cost already spent plus the cost of the work that is still planned.
RAF (reste a faire)	It is the number of man-days still to be worked, taken from the planned lines of the months that have not produced validated real days yet.
Earned Value (EV %)	It is the share of the contract that has really been delivered, judged and typed in by the project manager at the monthly review instead of being computed by the machine.
CA production	It is the revenue the company is allowed to treat as earned, equal to the value of the contract in TND multiplied by the EV percentage.
Jalon de facturation (billing milestone)	It is a slice of the contract, written as a percentage, that becomes an invoice amount of $\text{budget} \times \text{percentage} / 100$ and moves through the states PREVU, FACTURE and PAYE.
PPR (provision pour risques)	It is a risk cushion equal to 5% of the contract value in TND, shown on the project sheet as a reminder of how much room the project has before it is in trouble.

Traps to avoid

You might think `budgetPlanifie` is the money the client agreed to pay, but actually it is the internal cost of the whole workload plan ($\text{planned days} \times \text{daily rate} \times (1 + \text{TCC})$). The client's money is `budgetTnd`, which comes from the Project entity. So comparing `budgetPlanifie` with `budgetConsome` compares two costs. It does not compare a budget with a spend.

You might think EAC just adds the whole plan to the whole spend, but actually a planned line is dropped as soon as a validated real line exists for the same person and the same month - the filter is the text key `userId + ":" + period`. So if a manager forgets to validate a timesheet, that month is still counted as a forecast, and the EAC quietly stays too optimistic.

You might think the EV percentage is computed from the deliverables or from the days used, but actually a human types it into the snapshot form. `deliveryPct` (delivered deliverables / planned deliverables) is a separate indicator, and it is purely factual. And when no EV has ever been entered, the engine gives back null for CA production, FAE and current margin, and adds a warning instead of showing zero.

You might think a milestone amount follows the contract for ever, but actually only the milestones still in PREVU are priced again when an avenant changes the budget. A milestone that is already FACTURE or PAYE keeps its old amount, because the accounting for it is closed. That is exactly what the comment "les jalons FACTURE et PAYE sont geles" in `JalonService.recomputePrevuMontants` means.

You might think the KPI numbers live in the database, but actually the live endpoint computes all of them again from the raw days and rates on every GET. The `snapshot_kpis` table only keeps dated photographs taken at the monthly review, at most one per project per day. This is the opposite of the Excel workbook, and on purpose: there, an old stored value could quietly say something different from its own source cells.

Business modules and the rules they enforce (project, workload, billing, governance, mission, agile)

These modules store everything you need to follow a consulting project — its identity card and budget, who was planned to work on it, who really worked on it, when the client can be invoiced, what risks and deliverables exist, who travelled, and what the team still has to build. And they refuse any change that would break a company rule, for example invoicing more than 100 percent of the contract.

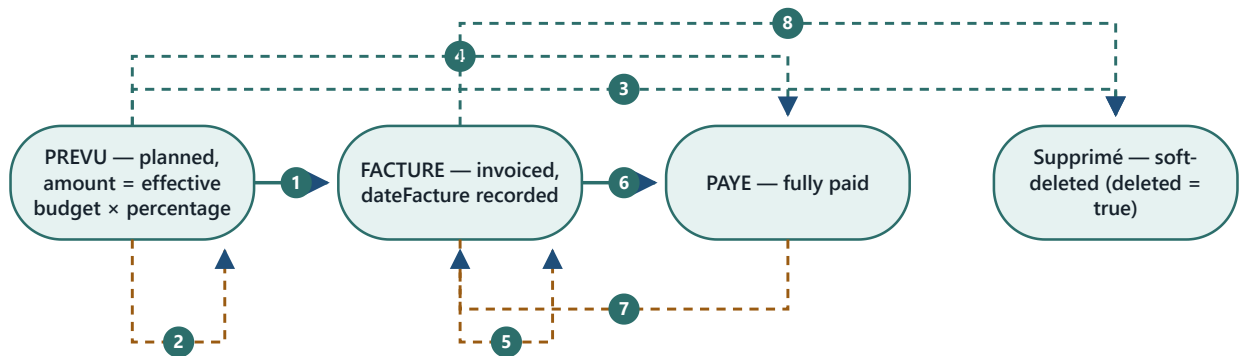
THINK OF IT LIKE...

Think of one building site. The project file holds the signed contract and its price. The plan de charge is the foreman's schedule pinned on the wall (who comes in which month). The charges réelles are the signed timesheets collected at the end of each month. The jalons are the payment slices written in the contract ("30 percent when the foundations are done"). An avenant is an extra order signed later that raises the total price. So every slice that has not been invoiced yet is suddenly worth more money — but the invoices already sent to the client are never touched.

WHY THIS EXISTS

ST2i used to follow each project in an Excel workbook: one sheet for the contract, one for the planned man-days, one for the real man-days, one for the invoicing milestones. Excel forces nothing. A colleague could type 40 percent + 40 percent + 40 percent of the contract into the milestone sheet. He could edit a milestone that had already been invoiced. He could delete a timesheet line that the margin calculation needed. Or he could forget to update the milestone amounts after an amendment to the contract. Each of those mistakes makes the project margin false, and nobody sees it happen. And that margin is the one number the company really manages by. The business modules turn each of those spreadsheet habits into a rule the server checks on every write. So the numbers behind the margin can be trusted, and several people can work on the same project at the same time without overwriting each other's file.

The picture



- 1 PREVU — planned, amount = effective budget × percentage → FACTURE — invoiced, dateFacture recorded
facturer(dateFacture) — JalonService refuses it from any status other than PREVU
- 2 PREVU — planned, amount = effective budget × percentage → PREVU — planned, amount = effective budget × percentage
update(), or recomputePrevuMontants() after an avenant — the amount is recalculated from the new budget
- 3 PREVU — planned, amount = effective budget × percentage → Supprimé — soft-deleted (deleted = true)
delete() — soft delete, allowed only while the milestone is still PREVU
- 4 PREVU — planned, amount = effective budget × percentage → PAYE — fully paid
BLOCKED — PaiementService refuses any payment while the milestone is PREVU ("le jalon doit être facturé avant d'enregistrer un paiement")
- 5 FACTURE — invoiced, dateFacture recorded → FACTURE — invoiced, dateFacture recorded
a partial payment is recorded; sum of payments is still below montant
- 6 FACTURE — invoiced, dateFacture recorded → PAYE — fully paid
recalculerStatut() — sum of non-deleted payments reaches or passes montant
- 7 PAYE — fully paid → FACTURE — invoiced, dateFacture recorded
a payment is deleted, the sum falls back below montant and dateFacture is set
- 8 FACTURE — invoiced, dateFacture recorded → Supprimé — soft-deleted (deleted = true)
BLOCKED — "impossible de supprimer un jalon déjà facturé ou payé"

Life of a billing milestone (jalon de facturation) — Notice that the human only starts the first arrow. The move to PAYE is decided by the code: it adds up the payments received and compares that total with the milestone amount. That is also why deleting a payment can send the milestone back to FACTURE.

Step by step

1

One project row is the backbone that everything else hangs on

A Project is the identity card of one signed contract. It holds a unique code, a name, the client, the funder (bailleur de fonds), the currency and its exchange rate to the Tunisian dinar, the start and end dates, the initial budget, the revised budget, the workload sold in man-days, the warranty days, the penalty provision, plus the director and the chef de projet (the project manager). Every other row in every other module points back to one project row by its id. That is why almost every service method starts with a projectId. Two small calculations sit right on the entity. `getEffectiveBudget()` gives back the revised budget when one exists, and falls back to the initial budget when there is none. `getPprTnd()` gives back five percent of the budget converted into dinars, the risk provision copied from the Excel identification sheet.

```
FILES backend/src/main/java/com/pms/project/entity/Project.java ·  
      backend/src/main/java/com/pms/project/service/ProjectService.java
```

2

The project also has a life, and it cannot jump backwards

A project moves through five states: DRAFT, ACTIVE, ON_HOLD, COMPLETED, CANCELLED. The moves that are allowed are written once, as a table inside the enum `ProjectStatus`. DRAFT can go to ACTIVE or CANCELLED. ACTIVE can go to ON_HOLD, COMPLETED or CANCELLED. ON_HOLD can come back to ACTIVE or be CANCELLED. COMPLETED and CANCELLED lead nowhere at all. `ProjectService.changeStatus` asks the current status whether the target is allowed, and refuses if it is not. So a finished project can never be re-opened by a careless click. Archiving is on purpose not a status but a separate true/false flag, and `ProjectService.archive` refuses to set it unless the project is COMPLETED.

```
FILES backend/src/main/java/com/pms/project/entity/ProjectStatus.java ·  
      backend/src/main/java/com/pms/project/service/ProjectService.java
```

3

Plan de charge: what the chef de projet expects to spend

A PlanCharge row says "this person is planned for this many days on this project, in this month". It holds the project, the user, a period (always stored as the first day of the month) and plannedDays. Only someone who holds the `VALIDATE_WORKLOAD` capability can create one, which in practice means the chef de projet. Two checks protect the plan. The person must already be an active member of the project team, or be the chef de projet himself. And there can be only one planned line per person per month, so the same month can never be counted twice. On a line that already exists, the month and the person can never be changed afterwards — only the number of days. That stops someone from quietly moving a plan from one month to another and changing the shape of the forecast.

```
FILES backend/src/main/java/com/pms/workload/service/PlanChargeService.java ·  
      backend/src/main/java/com/pms/workload/entity/PlanCharge.java
```

4

Charges réelles: what was really spent, and it freezes once it is validated

A ChargeReelle row has the same shape — project, user, month, actualDays — plus three audit fields: when it was submitted, when it was validated, and by whom. A consultant submits his own line. The rule assertOwnership means that anyone without the VALIDATE_WORKLOAD capability can only submit and edit lines for himself, never for a colleague. The chef de projet then validates the line, which stamps validatedAt and validatedBy. From that moment the line is frozen: editing it or deleting it is refused. That freeze is what makes the real figures an accounting record and not a draft.

```
FILES backend/src/main/java/com/pms/workload/service/ChargeReelleService.java ·
      backend/src/main/java/com/pms/workload/entity/ChargeReelle.java
```

5

Why we keep both the plan and the real figures: the gap between them is the margin

Neither table on its own answers the company's question. The KPI engine reads both. It multiplies the planned days by each person's daily cost to get budgetPlanifie. It multiplies the validated real days by the same daily costs to get budgetConsome. Then it builds the forecast for the end of the project: it keeps the real cost for the months that are already declared, and adds the planned cost only for the months that have no validated real line yet. Notice that it reads only validated real lines (findValidatedByProjectId), so a timesheet that is not validated never moves the margin. Without the plan there is no forecast and no early warning. Without the real figures there is no truth. The gap between the two is exactly what the old Excel workbook was computing by hand.

```
FILES backend/src/main/java/com/pms/kpi/service/KpiService.java ·
      backend/src/main/java/com/pms/workload/service/PlanChargeService.java ·
      backend/src/main/java/com/pms/workload/service/ChargeReelleService.java
```

6

Jalons de facturation: cutting the contract into slices you can invoice

A jalon de facturation is one invoicing milestone: a label such as "Livraison de la recette", a percentage of the contract, a money amount, a planned date, and a status. A human never types the amount — JalonService.computeMontant works it out as $\text{effective budget} \times \text{percentage} \div 100$, rounded to two decimals. Before any create or update, validatePourcentageSum adds up the percentages of all the project's milestones that are not deleted, takes away the milestone's own old percentage when this is an edit, adds the new one, and refuses the whole operation if the result would be more than 100. That is the rule that makes it impossible to plan to invoice a client more than the contract is worth.

```
FILES backend/src/main/java/com/pms/billing/service/JalonService.java ·
      backend/src/main/java/com/pms/billing/entity/JalonFacturation.java
```

7

A milestone goes PREVU, then FACTURE, then PAYE

A new milestone starts at PREVU (planned). Calling `facturer()` moves it to FACTURE (invoiced) and writes down the invoice date, and it is refused from any other status. While the milestone is still PREVU, `PaieementService` refuses to record a payment at all: the invoice has to exist before money can be attached to it. Each payment (an amount received, a date and a bank reference) triggers `recalculerStatut`. That method adds up all the payments of the milestone that are not deleted, and sets the status to PAYE as soon as the total reaches or passes the milestone amount. So partial payments leave the milestone at FACTURE. Editing and deleting are allowed only while the milestone is PREVU, because once it has been invoiced the accounting is closed.

FILES `backend/src/main/java/com/pms/billing/service/JalonService.java` ·
`backend/src/main/java/com/pms/billing/service/PaieementService.java` ·
`backend/src/main/java/com/pms/billing/entity/JalonStatut.java`

8

An avenant raises the budget, and the planned milestones follow it

An avenant is an amendment signed with the client: a number, a subject, an amount in money, and sometimes an impact in sold man-days. `AvenantService.create` does more than file the paper. It takes the current effective budget, adds the avenant amount, writes the result into `revisedBudget`, saves the project, and then calls `jalonService.recomputePrevuMontants(project)`. That method works out the money amount of every milestone still in PREVU again, from the new budget. It leaves FACTURE and PAYE milestones alone on purpose, because invoices already sent cannot change. Deleting an avenant does the mirror job: it takes the amount off the budget and works out the planned milestones again. The same recalculation runs from `ProjectService.update` when the initial budget is edited on a project that has no avenant yet.

FILES `backend/src/main/java/com/pms/billing/service/AvenantService.java` ·
`backend/src/main/java/com/pms/billing/service/JalonService.java` ·
`backend/src/main/java/com/pms/project/service/ProjectService.java`

Governance, missions and agile: the three side registers

Governance holds four registers attached to the project. Risk (a description, a probability and an impact, each rated FAIBLE/MOYEN/ELEVE, a mitigation plan, and an open/closed status). Livrable (a deliverable with a due date, which walks through EN_ATTENTE, EN_COURS, LIVRE, VALIDE, and the delivered ones feed the delivery percentage in the KPI). PartiePrenante (a stakeholder with a name, a role, a contact, an influence and an interest). And DemandeChangement (a change request with a requester, a priority and a decision date, which can only be approved, rejected, edited or deleted while it is still EN_ATTENTE). A Mission is a business trip for one team member on one project — subject, place, start date and end date. Each mission carries ComposanteMission cost lines typed PERDIEM, BILLET, TIMBRE, TRANSPORT or SEJOUR, each with its own amount and currency. The agile module holds the product backlog: BacklogItem (title, description, priority, estimate in man-days, TODO/IN_PROGRESS/DONE column, an assignee if there is one, a sprint if there is one) and Sprint (name, goal, start and end date, PLANNED/ACTIVE/CLOSED). An item with no sprint is simply still in the product backlog.

```
FILES backend/src/main/java/com/pms/governance/service/RiskService.java ·
      backend/src/main/java/com/pms/governance/service/LivrableService.java ·
      backend/src/main/java/com/pms/governance/service/PartiePrenanteService.java ·
      backend/src/main/java/com/pms/governance/service/DemandeChangementService.java ·
      backend/src/main/java/com/pms/mission/service/MissionService.java
```

The real code

backend/src/main/java/com/pms/billing/service/JalonService.java

JAVA

```
138     private void validatePourcentageSum(Long projectId, BigDecimal oldPct, BigDecimal newPct) {
139         BigDecimal current = jalonRepository.sumPourcentageByProjectId(projectId);
140         BigDecimal effective = current.subtract(oldPct != null ? oldPct :
141         BigDecimal.ZERO).add(newPct);
142         if (effective.compareTo(BigDecimal.valueOf(100)) > 0) {
143             throw new BusinessException(
144                 "La somme des pourcentages dépasserait 100% (actuel : " +
145                 current.toPlainString() + "%)");
146         }
147     }
148
149     private BigDecimal computeMontant(Project project, BigDecimal pourcentage) {
150         BigDecimal budget = project.getEffectiveBudget();
151         if (budget == null) return null;
152         return budget.multiply(pourcentage).divide(BigDecimal.valueOf(100), 2,
153             RoundingMode.HALF_UP);
154     }
```

What this does. BILLING RULE: the invoicing milestones of one project can never add up to more than 100 percent of the contract. Line 139 asks the database for the sum of the percentages of every milestone of this project that is not soft-deleted. Soft-deleted means marked deleted = true, while the row is still physically there. Line 140 computes what the sum would become: it takes away the milestone's own earlier percentage, which is null on a creation and the old value on an edit, then adds the new one — that subtraction is why changing a milestone from 40 to 45 percent is accepted instead of being counted twice. Line 141 compares that future sum with 100 using `compareTo`, which gives back a positive number only when the left side is strictly bigger, so reaching exactly 100 percent is allowed and 100.01 is refused. Throwing `BusinessRuleException` stops the whole database transaction, so nothing is saved. The second method shows that a user never types the money amount of a milestone: it is always the effective budget multiplied by the percentage and divided by 100, rounded to two decimals with `HALF_UP` (0.005 rounds up).

```
37     public AvenantResponse create(Long projectId, AvenantRequest request) {
38         Project project = loadProject(projectId);
39
40         // Mise à jour du budget révisé
41         BigDecimal currentBudget = project.getEffectiveBudget();
42         BigDecimal newRevisedBudget = (currentBudget != null ? currentBudget : BigDecimal.ZERO)
43             .add(request.montant());
44         project.setRevisedBudget(newRevisedBudget);
45         projectRepository.save(project);
46         jalonService.recomputePrevuMontants(project); // H-4
```

What this does. BILLING RULE: an avenant is a paper signed with the client that changes the contract. It changes the budget that the whole application works with, and the milestones that are not invoiced yet must follow it. Line 41 reads the effective budget. That is the revised budget when an earlier avenant already exists, and the initial budget in every other case. Lines 42 and 43 add the amount of the amendment to it. If the budget is missing, the code treats it as zero, so it never crashes on an empty project. Line 44 writes the result into revisedBudget. From that moment revisedBudget wins over initialBudget everywhere in the application, including in the milestone amount calculation shown above. Line 46 then asks the billing service to compute again the amount of every milestone that is still in PREVU. That method reads only the PREVU milestones. It leaves the FACTURE and PAYE ones alone, because an invoice already sent to the client must never change its value. Deleting an avenant runs the mirror code a few lines below: it takes the amount away and computes everything again.

```
101         if (cr.isValidated()) {
102             throw new BusinessException("Impossible de modifier une charge déjà validée");
103         }
104
105         // BR-033 : un développeur ne peut modifier que ses propres charges (même règle qu'à la
106 soumission)
107         assertOwnership(cr.getUser().getId());
108
109         LocalDate expectedPeriod = LocalDate.of(request.year(), request.month(), 1);
110         if (!cr.getPeriod().equals(expectedPeriod)) {
111             throw new BusinessException("La période d'une charge réelle ne peut pas être
112 modifiée");
113         }
114         if (!cr.getUser().getId().equals(request.userId())) {
115             throw new BusinessException("L'utilisateur d'une charge réelle ne peut pas être
116 modifié");
117         }
118     }
```

What this does. WORKLOAD RULE: a declared workload line, that is a line where a person writes the days he worked, becomes read-only the moment the chef de projet validates it. Line 101 calls `isValidated()`. On the entity that simply means `validatedAt` is not empty, and the code refuses the edit when it is not empty. The same guard also blocks deletion further down in the class. Line 106 calls `assertOwnership`. This is a private method. It throws `AccessDeniedException` (HTTP 403) when the current user does not hold the `VALIDATE_WORKLOAD` capability and is trying to touch somebody else's line. So a consultant can declare only his own days. Lines 108 to 111 build the month again from the request, as the first day of that month, and refuse the edit if it is not the same as the stored one: you cannot move a declaration from March to April. Lines 112 to 114 do the same thing for the person. Put together, these four checks mean one thing. An edit can change only the number of days, and only before validation.

```

200     @PreAuthorize("hasAuthority('EDIT_PROJECT')")
201     @Transactional
202     public ProjectResponse changeStatus(Long id, ProjectStatus newStatus) {
203         Project project = loadProject(id);
204         if (!project.getStatus().canTransitionTo(newStatus)) {
205             throw new BusinessException("Transition de statut interdite : "
206                 + project.getStatus() + " → " + newStatus);
207         }
208         project.setStatus(newStatus);
209         return toResponse(projectRepository.save(project));
210     }

```

What this does. PROJECT RULE: a project can move only to a status that its current status allows. Line 200 is a capability check. `@PreAuthorize` stops the call before the method body runs, unless the user who is logged in holds `EDIT_PROJECT`. The code checks a capability, never a role name. Line 201 marks the method transactional. That means every write inside it either all works or all is undone. Line 204 lets the status itself take the decision. `canTransitionTo` looks at a small table that is built one time inside the enum `ProjectStatus`. In that table, `DRAFT` allows `ACTIVE` and `CANCELLED`. `ACTIVE` allows `ON_HOLD`, `COMPLETED` and `CANCELLED`. `ON_HOLD` allows `ACTIVE` and `CANCELLED`. `COMPLETED` and `CANCELLED` allow nothing. Keeping the table inside the enum, and not in the service, means the same rule works everywhere the status is changed. Line 209 saves at last, and `toResponse` hides every money field from users who do not hold the `VIEW_KPI` capability.

```

83     @PreAuthorize("hasAuthority('MANAGE_GOVERNANCE')")
84     @Transactional
85     public LivableResponse valider(Long projectId, Long id) {
86         Livable livable = loadLivable(id, projectId);
87         if (livable.getStatut() != StatutLivable.LIVRE) {
88             throw new BusinessException("Seul un livrable livré peut être validé");
89         }
90         livable.setStatut(StatutLivable.VALIDE);
91         return livableMapper.toResponse(livableRepository.save(livable));
92     }

```

What this does. GOVERNANCE RULE: the client cannot accept a deliverable before it has been delivered. Line 86 loads the deliverable. Inside `loadLivable` it also checks that the deliverable really belongs to the project named in the URL. If it does not, the answer is "not found". That check stops somebody from reaching another project's data by guessing an id. Line 87 refuses the validation unless the deliverable is right now `LIVRE` (delivered). So nobody can jump over the chain `EN_ATTENTE`, `EN_COURS`, `LIVRE`, `VALIDE`. Line 90 does the single state change. The rest of the class finishes the rule in the other direction: once a deliverable is `VALIDE`, nobody can edit it or delete it any more. This matters for more than governance. The KPI engine computes the delivery percentage as the deliverables in `LIVRE` or `VALIDE` divided by the total number of deliverables.

```

66     @PreAuthorize("hasAuthority('MANAGE_AGILE')")
67     @Transactional
68     public void delete(Long projectId, Long id) {
69         Sprint sprint = loadSprint(id, projectId);
70
71         // Supprimer un sprint rend ses éléments au backlog produit plutôt que de
72         // les supprimer avec lui : le travail engagé mais non terminé n'est pas
73         // perdu, il redevient simplement non planifié. C'est aussi ce qui évite
74         // de laisser des éléments pointer, par clé étrangère, vers un sprint
75         // logiquement supprimé.
76         List<BacklogItem> committed = backlogItemRepository.findActiveBySprintId(id);
77         committed.forEach(item -> item.setSprint(null));
78         backlogItemRepository.saveAll(committed);
79
80         sprint.setDeleted(true);
81         sprintRepository.save(sprint);
82     }

```

What this does. AGILE RULE: deleting a sprint must never destroy the work that was committed to it. A sprint is a fixed time box, usually two or three weeks, into which the team pulls backlog items. Line 76 fetches every item that is not deleted and that is attached to this sprint right now. Line 77 sets the sprint field of each item to null. Line 78 saves them all at one time. Here null has a real meaning: it says the item is back in the product backlog, not planned. Only after that, on line 80, the sprint is marked deleted. And note that it is `setDeleted(true)`, a soft delete, so the row stays in the database for audit and history instead of going away. Detaching the items first is also what stops orphan rows, that is rows whose foreign key points at a sprint that no longer exists in the logic. The same module holds another rule in `BacklogItemService`: an item can be given only to somebody who is an active member of that project's team.

Words used in this chapter

Jalon de facturation (billing milestone)	A piece of the contract that you can bill when an agreed event happens, saved as a name, a percentage of the contract, a money amount that comes out of that percentage, and a status.
Plan de charge (planned workload)	The plan of how many man-days each team member should spend on a project in one given month, typed in by the chef de projet (the project manager).
Charge réelle (actual workload)	The number of man-days a team member says he really spent on a project in one given month, which can no longer be changed once the chef de projet validates it.
Avenant (contract amendment)	A paper signed with the client that changes the contract, and in the code it adds its amount to the project's revised budget and computes again every milestone that is still planned.
JH / jour-homme (man-day)	One person working one day, the unit used for planned workload, for real workload, and for backlog estimates.

Budget effectif (effective budget)	The number the whole application computes with: the revised budget when there is at least one avenant, and the initial budget when there is none.
Soft delete	Deleting a row by setting its deleted flag to true instead of wiping it out, so history and audits stay alive and all the queries filter on deleted = false.
BusinessRuleException	The project's own error type, thrown when a company rule is broken, which cancels the whole database transaction and sends an error message back to the screen.
Capability (authority)	A named permission such as <code>MANAGE_BILLING</code> or <code>VALIDATE_WORKLOAD</code> that the code checks before it runs a method, instead of checking a job title like "director".

Traps to avoid

You might think the amount of a billing milestone is typed in by the user, but actually only the percentage is typed. `JalonService.computeMontant` always works the amount out as $\text{effective budget} \times \text{percentage} \div 100$, and any amount that arrives in the request is thrown away.

You might think the 100 percent check forbids you to reach 100, but actually it uses `compareTo(...) > 0`, so a total of exactly 100 percent is accepted. And because it first takes away the milestone's own old percentage, you are allowed to change an existing milestone from 40 to 45 percent; the old value is not counted twice.

You might think an avenant is only a paper filed next to the project, but actually `AvenantService.create` rewrites `project.revisedBudget`. Then it computes again the amount of every milestone still in `PREVU`. Milestones that are already `FACTURE` or `PAYE` keep their old amount on purpose, because invoices already sent cannot change.

You might think `PAYE` is the end of the story, but actually deleting a payment calls `recalculerStatut` again. If the payments that are left no longer cover the milestone amount, the milestone goes back to `FACTURE`.

You might think `delete()` takes the row out of PostgreSQL, but actually almost every delete in these modules sets a deleted flag to true and saves the row. Every query filters on `deleted = false`. So the history is still there for an audit.

The Angular frontend (pms-frontend): routing, shell layout, signals, services, HTTP interceptor, and the project-picker pattern

It is the part of PMS that runs inside the user's web browser: it draws every screen, it chooses which screen the address bar should show, it adds the login token to every call it sends to the server, and it turns the answers back into tables, cards and forms.

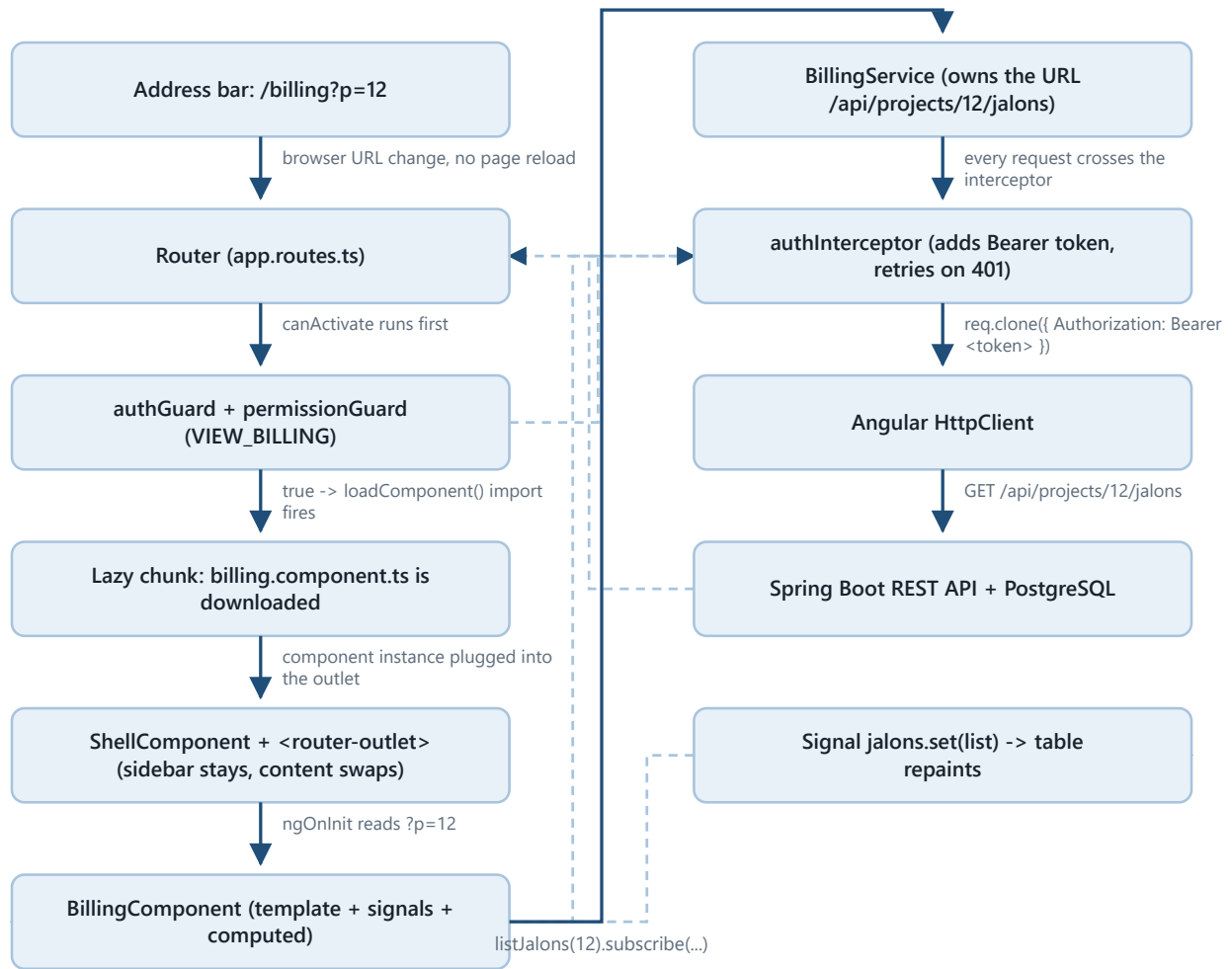
THINK OF IT LIKE...

Think of a hotel. The URL in the address bar is the room number a guest asks for. The router is the receptionist. He checks your key card first (those checks are the guards), and only then does he tell you the floor. He also unlocks and gets that one room ready only when someone really asks for it (this is lazy loading). The component is the room itself - the furniture, the lights and the way the room works, all in one place. The service is the room phone that calls the kitchen (the backend). The HTTP interceptor is the hotel operator. He stamps your room number on every call before it leaves the building, and if your key card ran out in the middle of a call, he quietly gets you a new one.

WHY THIS EXISTS

ST2i managed project margins in one shared Excel workbook. A workbook has no screens and no permissions, and many people cannot use it at the same time in a safe way. Whoever opened the file saw every sheet, every rate and every project. The Angular frontend exists so each role gets its own filtered view of the same numbers. A project manager sees his projects, a finance user sees billing milestones, and an admin sees roles. The real calculation still happens on the server. The frontend also replaces the workbook's row of tabs with a real navigation sidebar. And it replaces the workbook's VLOOKUP-style project lists with a project picker you can search, because the firm's project list (around 84 projects) is far too long to put in a dropdown.

The picture



From a typed URL to the database, and back — Look at where the interceptor sits. It sits on the arrow, not inside the component. No screen and no service ever touches the token itself. That is why you can add a new feature page and it can never forget by mistake to prove who the user is.

Step by step

1

The browser loads one single page, and Angular takes over

The server sends only one HTML file, `index.html`. Inside it there is one empty tag: `<app-root></app-root>`. The file `frontend/pms-frontend/src/main.ts` then starts Angular with `bootstrapApplication(App, appConfig)`. Angular finds that empty tag and fills it with the App component. After that, the browser never loads a full page again. Angular changes the content by itself. That is what "single page application" means: one HTML file, many screens.

FILES `frontend/pms-frontend/src/index.html` · `frontend/pms-frontend/src/main.ts` · `frontend/pms-frontend/src/app/app.ts`

2

The router reads the URL and chooses which screen to build

A router is a table. It links a piece of the URL to a component. In `app.routes.ts`, the text 'billing' is linked to `BillingComponent`. The App component's template is only this: `<router-outlet></router-outlet>`. Think of it as an empty socket, and the router plugs the matching screen into it. So when you type `/billing`, the browser does not ask the server for a page. It tells Angular "take out what is in the socket and put the billing screen there instead".

3

Two gates run before the screen is allowed to show

Each protected route lists `canActivate: [authGuard]` and `canActivate: [permissionGuard]`. A guard is a small function. It returns true (let the user in) or it returns a place to send the user instead. `authGuard` checks that a user session exists. If there is none, it sends the browser to `/login`. `permissionGuard` reads the permission name written in the route's data (for billing it is 'VIEW_BILLING'). It then checks that name against the permission list saved for the logged-in user. If the name is not there, it sends the user to `/dashboard` with `?forbidden=1`, so the dashboard can explain why. The sidebar uses the same permission names, so a user never even sees a menu entry he cannot open.

FILES `frontend/pms-frontend/src/app/core/guards/auth.guard.ts` · `frontend/pms-frontend/src/app/core/guards/permission.guard.ts` · `frontend/pms-frontend/src/app/layout/sidebar/sidebar.component.ts`

4

The screen's code is downloaded only at that moment (lazy loading)

The route does not write `component: BillingComponent`. It writes `loadComponent: () => import('./features/billing/billing.component')`. That small arrow function does not run when the app starts. The router calls it the first time someone visits `/billing`. So the build tool puts the billing screen in a separate JavaScript file. A user who never opens billing never downloads it. This keeps the first load of the application small, and that matters because PMS has around fifteen screens.

5

The component draws itself from signals

A component is one TypeScript class that holds three things together: the HTML template, the logic, and the CSS styles. In PMS all three are written inside the same `.ts` file. You can see this in the `@Component` decorator of `billing.component.ts`. Its data lives in signals. A signal is a box that holds a value and remembers who read it: `jalons = signal<JalonFacturation[]>([])`. You read it by calling it, `jalons()`, and you replace it with `jalons.set(list)`. When the value changes, Angular redraws only the parts of the template that read that signal. Nothing else moves. A `computed()` signal is a value made from other signals. `sortedJalons()` is one: it sorts itself again by itself whenever the list changes or the chosen sort column changes.

FILES `frontend/pms-frontend/src/app/features/billing/billing.component.ts`

6

The component asks a service, and the service speaks HTTP

A component must not know any URLs. It asks for a service instead. A service is a shared object, built once for the whole application, thanks to `@Injectable({ providedIn: 'root' })`. `BillingService` is the one that owns the addresses: `listJalons(projectId)` sends `GET /api/projects/{id}/jalons`, and `facturer(...)` sends `PATCH .../jalons/{id}/facturer`. The service gives back an `Observable`. An `Observable` is a promise of values that will come later. Nothing is really sent until the component calls `.subscribe(...)`. The function inside `subscribe` writes the answer into a signal, and that redraws the table.

FILES `frontend/pms-frontend/src/app/core/services/billing.service.ts` · `frontend/pms-frontend/src/app/features/billing/billing.component.ts`

7

The interceptor stamps the request on its way out

An HTTP interceptor is a function placed on the pipe that every request travels through. It is set up one time in `app.config.ts` with `provideHttpClient(withInterceptors([authInterceptor]))`. In Angular you cannot change a request after you make it, so the interceptor makes a copy of it and adds the header `Authorization: Bearer <token>`. It also watches the answers that come back. On HTTP 401 (the token has expired) it calls the refresh endpoint one time, then it sends the first request again by itself with the new token, so the user never sees an error. If the refresh fails, it clears the session and goes back to `/login`.

FILES `frontend/pms-frontend/src/app/app.config.ts` · `frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts`

8

A small pop-up form sends the change back, and the screen loads its data again

When you click "Ajouter un jalon", the code sets a signal, `modal.set('jalon')`, and the `@if` block in the template makes the dialog show up. Each input is tied to a field of a plain object with `[(ngModel)]`. This is the template-driven form style, where the HTML holds the shape of the form. `saveJalon()` sends nothing if the label or the percentage is empty. It shows a spinner by setting `saving.set(true)`, then it calls the service. When it works, it calls `reload()` to get the list again from the server. That way any value the backend computed, like the amount of the jalon in TND, is the server's value and not a guess. Then it shows a green toast. When it fails, it shows the message the backend sent back.

FILES `frontend/pms-frontend/src/app/features/billing/billing.component.ts`

The real code

frontend/pms-frontend/src/app/app.routes.ts

TYPESCRIPT

```
89     {
90         path: 'billing',
91         title: 'Facturation',
92         canActivate: [permissionGuard],
93         data: { permission: 'VIEW_BILLING' },
94         loadComponent: () => import('./features/billing/billing.component').then(m =>
95             m.BillingComponent)
96     },
```

What this does. This is one row of the routing table, and it answers four questions at the same time. `path: 'billing'` means this row matches the address `/billing`. `title` sets the text you see in the browser tab. `canActivate: [permissionGuard]` names a function that must say yes before anything is shown, and `data: { permission: 'VIEW_BILLING' }` is the value that function will read. So the same guard code works for every screen, and only the permission name changes. `loadComponent` holds an arrow function with `import(...)` inside it. That is a dynamic import: the billing screen's code is fetched from the server only when a user first opens this address. That is exactly what lazy loading means.

frontend/pms-frontend/src/app/core/guards/permission.guard.ts

TYPESCRIPT

```
5 export const permissionGuard: CanActivateFn = (route: ActivatedRouteSnapshot) => {
6     const auth = inject(AuthService);
7     const router = inject(Router);
8     const required: string = route.data['permission'];
9     if (!required || auth.hasPermission(required)) return true;
10    // M-8: communicate why the user was redirected so the dashboard can show a message
11    return router.createUrlTree(['/dashboard'], { queryParams: { forbidden: '1' } });
12 };
```

What this does. A guard here is a plain function, not a class. `inject(AuthService)` is dependency injection. Instead of building the service itself, the function asks Angular for the one shared copy that already holds the logged-in user. `route.data['permission']` takes the text that the route wrote next to it, for example `'VIEW_BILLING'`. If the route asked for nothing, or if the user's permission list has that name in it, the function returns true and the screen is built. If not, it returns a URL object that points at `/dashboard` with `?forbidden=1`, and the router goes there right away. The redirect carries a reason, so the dashboard can say why instead of just pushing the user away without a word.

```
7 export const authInterceptor: HttpInterceptorFn = (req: HttpRequest<unknown>, next: HttpHandlerFn)
8 => {
9   const auth = inject(AuthService);
10  const router = inject(Router);
11
12  const token = auth.getAccessToken();
13  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
14
15  return next(authReq).pipe(
16    catchError((err: HttpErrorResponse) => {
17      // H-2: server-side first-login gate returned 403 with FIRST_LOGIN_REQUIRED
18      if (err.status === 403 && err.error?.code === 'FIRST_LOGIN_REQUIRED') {
19        router.navigate(['/change-password']);
20        return throwError(() => err);
21      }
22
23      const isAuthEndpoint = req.url.includes('/auth/refresh') ||
24      req.url.includes('/auth/logout');
25      if (err.status === 401 && !isAuthEndpoint) {
26        // H-1: handleTokenRefresh() ensures only one /auth/refresh call goes out at a time.
27        // Concurrent 401 callers subscribe to the shared BehaviorSubject and all get
28        // the same refreshed token without firing duplicate refresh requests.
29        return auth.handleTokenRefresh().pipe(
30          switchMap(res => {
31            const retried = req.clone({ setHeaders: { Authorization: `Bearer ${res.accessToken}` } }
32          ));
33          return next(retried);
34        });
35      }
36    })
37  );
```

What this does. The function gets req, the request going out, and next, the rest of the pipe. You cannot change a request object, so req.clone({ setHeaders: ... }) makes a copy that has the header Authorization: Bearer <token>. If there is no token, the first request goes out as it is, and that is what lets the login call work. next(authReq) sends it on, and .pipe(catchError(...)) looks at whatever comes back. The first check catches a 403 whose body has the code FIRST_LOGIN_REQUIRED, and it pushes the user to the change-password screen. The server decides; the browser only does as it is told. The second check catches 401 (the token has expired), but on purpose it skips the refresh and logout endpoints, because asking for a refresh after a failed refresh would loop for ever. switchMap means "when the refresh is done, drop its result and run this new request instead". The first call is built again with the fresh token and sent again, so the user's click works and he never sees an error.

```
11 private base(projectId: number) {
12     return `${environment.apiUrl}/projects/${projectId}`;
13 }
14
15 listJalons(projectId: number): Observable<JalonFacturation[]> {
16     return this.http.get<JalonFacturation[]>(`${this.base(projectId)}/jalons`);
17 }
18
19 facturer(projectId: number, jalonId: number, dateFacture: string): Observable<JalonFacturation>
20 {
21     return this.http.patch<JalonFacturation>(
22         `${this.base(projectId)}/jalons/${jalonId}/facturer`,
23         { dateFacture }
24     );
25 }
```

What this does. environment.apiUrl is the one place where the server address is written:

http://localhost:8090/api while you develop, and just /api once the app is deployed. So no URL has to be edited before going live. base(projectId) builds the common start of the address one time instead of repeating it in every method. listJalons returns Observable<JalonFacturation[]>, which means "a future list of billing jalons". The angle brackets are only a TypeScript promise about the shape of the data; the server is still the one that has to keep that promise. facturer uses PATCH, the HTTP verb for changing part of a record that already exists, and it sends { dateFacture } as the body. The business rule that moves the jalon from PREVU to FACTURE lives on the server, not here. Look again: no token shows up anywhere in this file. That is the interceptor's job.

```
519 saveJalon(): void {
520     if (!this.jalonForm.label || !this.jalonForm.pourcentage) {
521         this.modalError.set(this.tr.translate('billing.errLabelPct'));
522         return;
523     }
524     this.saving.set(true);
525     this.modalError.set('');
526     const body = {
527         label: this.jalonForm.label,
528         pourcentage: this.jalonForm.pourcentage,
529         datePrevue: this.jalonForm.datePrevue || undefined
530     };
531     this.billingSvc.createJalon(this.selected()!.id, body).subscribe({
532         next: () => { this.reload(); this.modal.set(null); this.saving.set(false);
533         this.toast.success(this.tr.translate('billing.okMilestoneAdded')); },
534         error: (e) => { this.modalError.set(e.error?.message ?? 'Erreur. '); this.saving.set(false);
535     }
    });
}
```

What this does. The first if is a cheap check done here in the browser. With no label or no percentage, nothing is sent, and an error message is put into a signal that the template shows. `saving.set(true)` flips a signal that turns off the Save button and shows a spinner. This stops a double click from creating two jalons. An empty `datePrevue` is turned into `undefined`, so the JSON simply leaves the field out instead of sending an empty string that the backend would have to work out. `.subscribe({ next, error })` is the point where the request really goes out. When it works, `reload()` gets the list again, so the amount in TND on the screen is the one the server computed from the percentage; `modal.set(null)` closes the dialog, and a green toast says it is done. When it fails, the component shows `e.error.message`. Those are the backend's own words, for example the refusal when the jalon percentages would go over 100, instead of text the frontend made up.

```

375     readonly filtered = computed(() => {
376         const s = this.search().toLowerCase().trim();
377         const fav = this.favorites();
378         return this.projects().filter(p => {
379             if (this.onlyMine() && p.chefProjetId !== this.myId) return false;
380             if (this.onlyFav() && !fav.has(p.id)) return false;
381             if (!s) return true;
382             return p.code.toLowerCase().includes(s)
383                 || p.name.toLowerCase().includes(s)
384                 || (p.client ?? '').toLowerCase().includes(s)
385                 || (p.chefProjetName ?? '').toLowerCase().includes(s);
386         });
387     });

```

What this does. `computed()` creates a value that builds itself again. It reads four signals - `search()`, `favorites()`, `onlyMine()`, `onlyFav()` - and Angular writes down that link. So typing one more letter in the search box runs this function again, and only this function, and then it repaints the table. The two early returns are the filter chips: "Assignés" keeps only the projects whose `chefProjetId` is the same as my own user id, and "Favoris" keeps only the starred ones. If the search box is empty, the project passes. If not, the text is matched against four fields at once - code, name, client and project manager. So the user can type "Tunisie Telecom" or "PRJ-014" or the name of a colleague and find the same project. The `??` protects against a missing client, because calling `toLowerCase()` on null would break the whole screen.

Words used in this chapter

Component	One class that holds a screen's three parts together - the HTML template that is drawn, the TypeScript logic that answers clicks, and the CSS styles - which in PMS are all written inside one single .ts file under the <code>@Component</code> decorator.
Standalone component	A component that lists what it needs in its own imports: [...] line instead of being listed in one big shared NgModule file, which is why PMS has no *.module.ts files at all and every component says exactly what it uses.
Signal	A box that holds a value and remembers which parts of the screen read it, so calling <code>.set()</code> or <code>.update()</code> on it redraws only those parts - and you read it by calling it like a function, for example <code>jalons()</code> .
Computed signal	A read-only signal whose value is worked out from other signals and worked out again by itself when any of them changes, such as <code>sortedJalons()</code> or <code>filtered()</code> .
Lazy loading	Downloading a screen's JavaScript only the first time a user goes to it, written in PMS as <code>loadComponent: () => import('...')</code> inside the route.
Route guard	A function the router runs before a screen shows up, which returns true to let it through or a redirect URL to send the user somewhere else - PMS uses <code>authGuard</code>

(are you logged in), `permissionGuard` (do you have this permission) and `unsavedChangesGuard` (are you about to lose your edits).

HTTP interceptor	A function set up one time that every request and every answer goes through, used in PMS to add the Authorization header and to quietly get a new token when the old one has expired.
Observable / subscribe	An Observable is a plan for work that will give you values later; nothing is really sent to the server until someone calls <code>.subscribe()</code> , and the function you give to subscribe gets the answer.

Traps to avoid

You might think the guards are what keep the application safe, but actually they only protect the user interface. A guard runs inside the browser, and anyone can edit browser code. So `permissionGuard` hiding `/admin/users` is a help, not a lock. The real protection is the matching check on the Spring Boot side. The frontend guard is there so users are not shown doors that will shut in their face. Say this plainly if the jury asks. It shows you know where the line of trust is.

You might think PMS uses reactive forms, but actually it does not. If you search the whole `src` folder for `FormBuilder`, `FormGroup`, `FormControl` or `ReactiveFormsModule`, you find nothing. Every form in the project is template-driven: `FormsModule` plus `[(ngModel)]` tied to a plain object like `jalonsForm = { label: "pourcentage: 0, datePrevue: " }`, with the rules written as if statements at the top of `saveJalons()`. Reactive forms are the other Angular style. There you build the form in TypeScript as a tree of `FormControl` objects with checks attached. That style is easier to unit-test and better for long forms or forms that change shape. Here is the honest defence answer: the PMS forms are short pop-ups with two or three fields, and the backend has to check the rule again anyway, so the lighter style was enough. On a longer form, like creating a project, reactive forms would have been the better choice.

You might think the UI updates because Angular checks everything again after each click, but actually this build has no `zone.js` in `package.json` at all, so nothing is watching the whole application. The screen updates only because a signal was written: `this.jalons.set(d)` redraws the table of `jalons`. If you set a plain field instead of calling `.set()` on a signal, the screen will simply not move. That is the single most common bug in this style of code.

You might think a component must be listed in a module before you can use it, but actually PMS has no `NgModule` file anywhere. Components are standalone and say what they need by themselves, in imports: `[CommonModule, FormsModule, ProjectPickerComponent, TranslocoModule]`. Note too that `App` (`app.ts`) does not even write `standalone: true`. In recent Angular versions `standalone` is the default and the flag is not needed. The `standalone: true` you still see on the older components is just left over from before.

You might think `/billing` and `/billing?p=12` are two different routes, but actually they are the same route. The router does not look at the query string after the `?`; the component reads it. `BillingComponent` listens to `route.queryParamMap`, and `select()` calls `router.navigate([], { queryParams: { p: p.id } })`. That one design choice is why a user can bookmark the billing page of one project, or send the link to a colleague. It is also why the browser Back button walks back through the projects he opened instead of leaving the screen.

Build, test and delivery pipeline (Docker, nginx, GitHub Actions, Maven/JaCoCo)

It packs the whole application — database, API and web interface — into three sealed boxes that start with one single command, and it runs an automatic checklist on every code change, so a broken version never reaches a server.

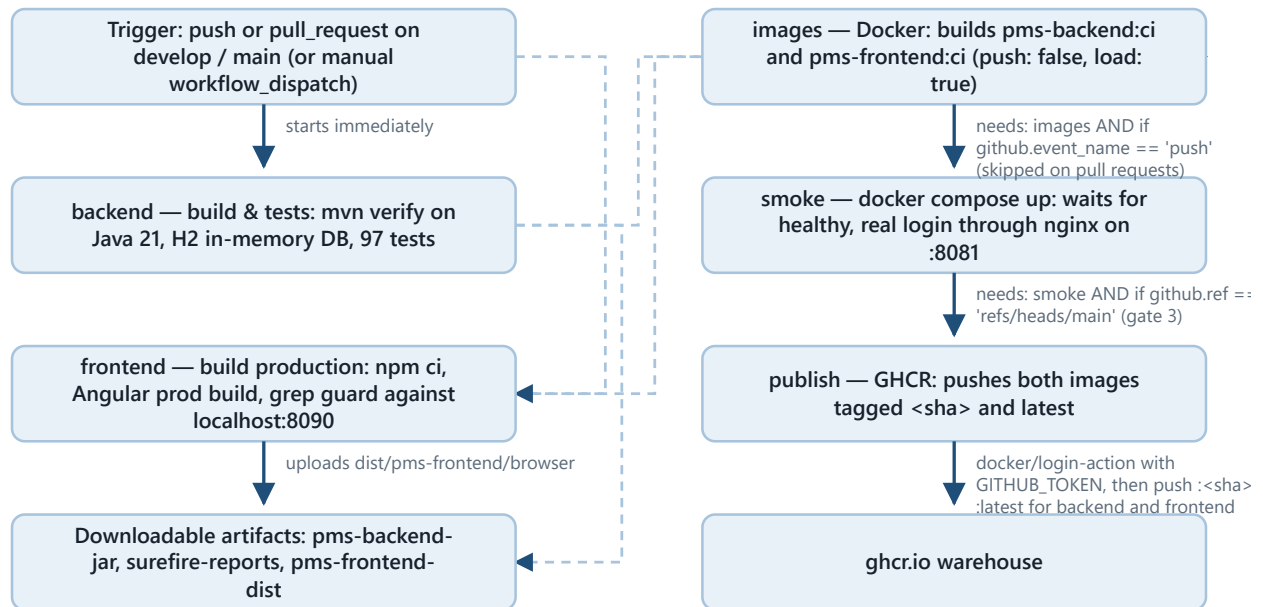
THINK OF IT LIKE...

Think of a central kitchen that makes sealed meal trays, ready to heat. The recipe and every ingredient are put into one tray at the factory (that tray is the "image"). A tray that is really being heated and eaten in someone's kitchen is a "container". Because the tray brings its own plate, its own sauce and its own cooking temperature, the meal tastes the same in Tunis, on the examiner's laptop, or on ST2i's server. The CI pipeline is the inspection line at the end of the factory: every tray is weighed, tasted and X-rayed at separate stations, and only the trays that pass every station get a label and go into the warehouse.

WHY THIS EXISTS

PMS replaces an Excel workbook that ST2i passed around by email. An Excel file needs nothing to run: you double-click it and it works. A Spring Boot API plus an Angular app plus PostgreSQL normally need the right Java version, the right Node version, a database installed with the right user and password, and four free ports. If installing PMS were harder than opening the old workbook, nobody at ST2i would change. So the project ships as three containers driven by one `docker-compose.yml`: `cp .env.example .env`, then `docker compose up`, then open `http://localhost:8081`. The second reason is trust in the numbers. The old workbook computed project margins. If PMS quietly breaks a margin formula, ST2i loses money and nobody sees it. The CI pipeline runs 97 automated checks again on every push. It even starts the whole stack and logs in for real, so a machine catches the problem instead of an accountant.

The picture



The PMS CI pipeline: five jobs, three gates — Look at three things here. The two build jobs run side by side, and both must pass before any image is built. The slow end-to-end smoke test is skipped on pull requests, to keep review fast. And nothing is ever published unless the smoke test passed on the main branch.

Step by step

1

A developer pushes code, and GitHub wakes up the pipeline

CI means "continuous integration": a server builds and checks the project again by itself every time the code changes, instead of the developer having to remember to do it. The file `.github/workflows/ci.yml` tells GitHub when to run. It runs on any push to the branches `develop` and `main`, on any pull request (a proposal to merge code) aimed at those branches, and on a manual click (`workflow_dispatch`). A `concurrency` block with `cancel-in-progress` means that if you push twice very quickly, the older run is stopped, so the machines are not wasted.

FILES `.github/workflows/ci.yml`

2

The `backend` job compiles Java and runs the 97 tests

This job installs Java 21 and runs `mvn -B -ntp verify` inside the backend folder. `verify` is the Maven command that compiles the code, runs every test, and fails the whole job if even one test fails. The tests do not need a real PostgreSQL server. They use H2, a small database that lives only in memory, set up in `backend/src/test/resources/application-test.yml` with

`jdbc:h2:mem:testdb;MODE=PostgreSQL`. Flyway (the tool that applies the numbered SQL migration files) is switched off there, and Hibernate makes the tables straight away. At the end, the job uploads two things you can download from the GitHub page: the test reports and the built JAR file (a JAR is one single `.jar` file that holds the whole compiled Java application).

FILES `.github/workflows/ci.yml` · `backend/src/test/resources/application-test.yml` · `backend/pom.xml`

3

The `frontend` job builds the Angular bundle and searches it for one mistake

This job installs Node 22 and runs `npm ci`, which installs exactly the versions written in `package-lock.json` instead of picking newer ones. Then it builds the production bundle. There is no `npm test` step, and that is on purpose. The workflow explains why in a comment: `angular.json` declares no test target, and `karma/jasmine` are not installed, so adding the step would make CI red for ever. Instead the job adds a check made by hand: it searches the compiled JavaScript for the text `"localhost:8090"`. If that development URL is still in there, it means the Angular environment file was not swapped for the production one. The app in the container would then call the developer's own machine instead of the backend container, so the job fails on purpose.

FILES `.github/workflows/ci.yml` · `frontend/pms-frontend/package.json` · `frontend/pms-frontend/angular.json`

4

The `images` job builds the two Docker images

This job waits until both `backend` and `frontend` have passed — that is what `needs: [backend, frontend]` means. Then it builds the two Docker images with `buildx`. `push: false, load: true` means the images are built and kept on the runner but not uploaded anywhere. This job only proves that the Dockerfiles still work. `cache-from / cache-to type=gha` keeps the in-between build layers in GitHub's cache, so the next run does not have to download every Maven and npm dependency again.

FILES `.github/workflows/ci.yml` · `backend/Dockerfile` · `frontend/pms-frontend/Dockerfile`

5

The `smoke` job starts the whole stack and logs in for real

A smoke test is a quick end-to-end check that the finished product turns on at all. This job runs only on a push, not on a pull request, because it is slow. It copies `.env.example` to `.env`, replaces the JWT secret and the database password with random throwaway values, and runs `docker compose up -d --build`. Then it keeps asking `docker inspect` until the backend container says its status is "healthy". Then comes the real proof: it POSTs a login to `http://localhost:8081/api/auth/login` and asks for HTTP 200 plus an `accessToken` in the answer. Port 8081 is nginx, so that one request travels from browser to nginx to Spring Boot to PostgreSQL. It also checks that a deep URL like `/projects` returns 200 (Angular routing fallback), and that `/i18n/fr.json` is served (the translation files). `docker compose down -v` always runs at the end, even when something failed.

FILES `.github/workflows/ci.yml` · `docker-compose.yml` · `.env.example`

6

The `publish` job uploads the images, but only from `main`

This last job has two locks. First, `needs: [smoke]`, so it never runs if the end-to-end test failed. Second, an `if` condition that asks for the branch to be exactly `refs/heads/main` and the event to be a push. It logs in to GHCR (GitHub Container Registry, GitHub's warehouse for Docker images) using a token GitHub makes for the run. Then it pushes each image twice: once tagged with the exact commit hash (`{{ github.sha }}`), so any old version can be deployed again, and once tagged `latest`. To deploy to a real server you then just pull those two tags — nothing is compiled on the server.

FILES `.github/workflows/ci.yml`

7

We build each image in two stages, so the image we ship stays small

A Dockerfile is the recipe for an image. Both Dockerfiles here are "multi-stage". That means the first stage holds the big, heavy tools, and the second stage holds only the result. For the backend, stage 1 is a Maven + JDK image that builds the JAR. Stage 2 is a plain Java runtime (JRE) image. It gets only that one JAR, through `COPY --from=build`. Maven, the source code and the ~200 MB cache of dependencies stay behind in stage 1 and never ship. For the frontend, stage 1 is a Node image that builds the static HTML/JS/CSS files. Stage 2 is an nginx image that gets only the `dist` folder, so Node never runs in production. The backend image also creates a user who is not root (`uid 1001`) and switches to it. So if someone breaks into the container, he is not root right away.

FILES `backend/Dockerfile` · `frontend/pms-frontend/Dockerfile` · `backend/.dockerignore` · `frontend/pms-frontend/.dockerignore`

JaCoCo measures how much of the code the tests really touch

Counting tests does not tell you what they reach. So `backend/pom.xml` adds the JaCoCo plugin (lines 194-213). Its `prepare-agent` goal puts an agent inside the test JVM, and that agent writes down every bytecode instruction that runs. Its `report` goal is tied to the `test` phase and writes `backend/target/site/jacoco`. Here are the real numbers from the last run on disk: 97 tests across 13 test classes, all passing, 31.9% instruction coverage, 41.2% line coverage, 24.5% branch coverage. That headline number is pulled down by about 11,000 instructions of demo-data seeders (`EnterpriseDataSeeder`, `DemoDataSeeder`, `AgileDemoSeeder`, `DataInitializer`), which only insert sample rows. If you leave those out, coverage is 53.7% of instructions and 56.5% of lines. The weakest real modules are `com.pms.mission` at 0% and `com.pms.user.service` at 2.8%. Note also that `docs/DEPLOYMENT.md` still says "89 tests" — that line is old, and the true count today is 97.

FILES `backend/pom.xml` • `docs/DEPLOYMENT.md`

The real code

backend/Dockerfile

DOCKERFILE

```

7  # — Étape 1 : construction —————
8  FROM maven:3.9-eclipse-temurin-21 AS build
9  WORKDIR /build
10
11 # Couche de dépendances : invalidée uniquement si pom.xml change.
12 # Si cette étape échoue (annotationProcessorPaths Lombok/MapStruct), on peut
13 # supprimer ces deux lignes : on perd le cache, rien d'autre.
14 COPY pom.xml .
15 RUN mvn -B -ntp dependency:go-offline
16
17 # Couche applicative
18 COPY src ./src
19 # Les tests sont exécutés par la CI, pas ici : construire l'image est une étape
20 # d'empaquetage, pas la barrière qualité. Les rejouer doublerait le temps de build
21 # sans apporter de signal supplémentaire.
22 RUN mvn -B -ntp clean package -DskipTests
```

What this does. `FROM maven:3.9-eclipse-temurin-21 AS build` starts a temporary build box that already has Maven (the Java build tool) and a full Java 21 development kit inside. `AS build` gives that stage the name "build", so a later stage can reach into it and take files out. The pair `COPY pom.xml .` then `RUN mvn dependency:go-offline` is a caching trick on purpose. Docker remembers each instruction as a layer, and it runs a layer again only when the inputs of that layer change. So the hundreds of Java dependencies are downloaded again only when `pom.xml` itself changes, not every time you change a line of code. `COPY src ./src` brings the real source code after that, exactly so that editing the source does not throw away the dependency layer. The last line packs the application into a JAR with `-DskipTests`, and the comment says why: building the image is packaging, not quality control, because the CI already ran the tests. Running them twice would double the build time and tell us nothing new.

```
31      # — Reverse proxy vers le backend —————
32      # `^~` : ce préfixe l'emporte sur la location regex des fichiers statiques.
33      # Pas d'URI après le host dans proxy_pass ⇒ le chemin /api/... est transmis tel quel.
34      location ^~ /api/ {
35          proxy_pass          http://backend:8080;
36          proxy_http_version 1.1;
37          proxy_set_header    Host            $host;
38          proxy_set_header    X-Real-IP       $remote_addr;
39          proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
40          proxy_set_header    X-Forwarded-Proto $scheme;
41          proxy_read_timeout  60s;
42      }
```

What this does. A `location` block tells nginx what to do with URLs that start with a given path. Here, any address that starts with `/api/` is not looked for on disk. `proxy_pass http://backend:8080` passes the request on to the Spring Boot container, and Docker's own internal network finds that container by its service name "backend". Everything else in the file is sent back as a file from the Angular build. The result is the important part. The browser only ever talks to one address, `http://localhost:8081`. So the page and the API have the same "origin" (protocol + host + port), and the browser's cross-origin rules (CORS) simply never apply. The `proxy_set_header X-Real-IP` and `X-Forwarded-For` lines pass the visitor's real IP address along, because otherwise the backend would think every request came from nginx. The `^~` mark is a priority rule. It makes this block win over the regular-expression block further down that matches `.js` and `.css` files, so an API path is never taken for a static file.

```
93     restart: unless-stopped
94     depends_on:
95         backend:
96             # nginx résout « backend » une seule fois au démarrage : si le conteneur
97             # n'existe pas encore, il s'arrête sur « host not found in upstream ».
98             condition: service_healthy
99     ports:
100     - "${FRONTEND_HOST_PORT:-8081}:80"
101     healthcheck:
102     test: ["CMD", "wget", "-qO-", "http://127.0.0.1/healthz"]
103     interval: 10s
104     timeout: 3s
105     retries: 5
106     start_period: 10s
```

What this does. A healthcheck is a small command that Docker runs inside the container again and again on a timer. It decides whether the program inside is really working, and not just started. Here Docker asks nginx for /healthz every 10 seconds, waits 3 seconds for an answer, and marks the container unhealthy after 5 failures in a row. `start_period: 10s` gives it 10 seconds of grace at boot, and failures in that time do not count. `depends_on: backend: condition: service_healthy` is the choice that makes the stack start in the right order: the frontend container is not even created until the backend's own healthcheck has gone green. The comment explains why this is not optional. nginx looks up the name "backend" one single time at startup, and it refuses to start with "host not found in upstream" if that container is not there yet. `restart: unless-stopped` asks Docker to start the container again after a crash or after the machine reboots, unless a human stopped it on purpose. `"${FRONTEND_HOST_PORT:-8081}:80"` maps port 80 inside the container to port 8081 on the host, and uses 8081 as the default when the variable is not set.

```
84     # Garde-fou : la configuration production doit avoir substitué
85     # environment.prod.ts. Si l'URL de développement réapparaît dans le bundle,
86     # le frontend conteneurisé appellerait le backend de l'hôte.
87     - name: Vérifier la substitution d'environnement
88       working-directory: frontend/pms-frontend
89       run: |
90         if grep -rq "localhost:8090" dist/pms-frontend/browser/*.js; then
91           echo "::error::L'URL de développement est présente dans le bundle de production
92 (fileReplacements cassé).\"
93         exit 1
94       fi
95       echo \"OK – aucune URL de développement dans le bundle.\"
```

What this does. This is a safety check we wrote by hand, and it is more interesting than any normal build step. Angular's production setup swaps `environment.ts` for `environment.prod.ts`. That swap changes the API address from `http://localhost:8090` (the developer's own local Spring Boot) to `/api` (the same origin as nginx). If that swap ever breaks, the build still succeeds and nothing looks wrong. But the app we ship would call the developer's own laptop instead of the backend container, and every user would get connection errors. So the job simply searches the compiled JavaScript for the forbidden text with `grep -rq`. `exit 1` makes the job fail, and that stops the whole pipeline. The `::error::` prefix is GitHub Actions syntax. It turns the message into a red note on the run page instead of an ordinary log line.

```
161     # Preuve que la chaîne complète fonctionne : navigateur → nginx → API → base.
162     - name: Connexion via le reverse proxy nginx
163       run: |
164         code=$(curl -s -o /tmp/login.json -w '%{http_code}' \
165           -X POST http://localhost:8081/api/auth/login \
166           -H 'Content-Type: application/json' \
167           -d '{"email":"admin@pms.local","password":"Admin1234!"}')
168         echo \"HTTP $code\"; cat /tmp/login.json
169         test \"$code\" = \"200\"
170         grep -q 'accessToken' /tmp/login.json
```

What this does. This is the proof that the whole chain works from end to end, and it is the step worth showing at the defence. `curl` sends a real login request to port 8081 — the nginx container — not straight to the backend. For this to come back 200, five different things must all be right at the same time: nginx started, its `/api/` proxy rule points at the right container, Spring Boot is up, Flyway applied all its migrations to PostgreSQL, and the demo admin account was created with exactly that password. `-w '%{http_code}'` takes only the HTTP status and puts it into the shell variable `code`. `test \"$code\" = \"200\"` makes the job fail on anything else. The last `grep -q 'accessToken'` checks that the reply really has a token inside, because an empty 200 would still pass the status check. Note that the password is a published demo password from `docs/demo-accounts.md`, not a real secret.

```

44      // — Percentage sum validation —————
45
46      @Test
47      @DisplayName("create — somme des pourcentages > 100 % lève BusinessException")
48      void create_pourcentageSumExceeds100_throws() {
49          Project project = stubProject(1L, BigDecimal.valueOf(100_000));
50          when(projectRepository.findActiveById(1L)).thenReturn(Optional.of(project));
51          when(jalonRepository.sumPourcentageByProjectId(1L)).thenReturn(BigDecimal.valueOf(80));
52
53          // New jalon at 30% would push sum to 110%
54          JalonRequest req = new JalonRequest("Jalon 3", BigDecimal.valueOf(30), LocalDate.now());
55          assertThatThrownBy(() -> service.create(1L, req))
56              .assertInstanceOf(BusinessRuleException.class)
57              .hasMessageContaining("100%");
58      }

```

What this does. This is what a unit test looks like: one small piece of logic checked on its own, with no database and no web server. The class is annotated `@ExtendWith(MockitoExtension.class)` and its repositories are `@Mock`. A mock is a fake stand-in, and the test decides what it answers. The two `when(...).thenReturn(...)` lines set up the situation: the project exists, and the billing milestones it already has take up 80% of the contract. Then the test asks the service to add a milestone worth 30%, which would bring the total to 110%. `assertThatThrownBy` says what must happen: the service has to refuse with a `BusinessRuleException` whose message talks about "100%". The test just below it proves that the limit itself is allowed: 70% + 30% = exactly 100% is accepted and must not throw. The method name and the `@DisplayName` both say the rule in words, so a failing test report reads like a sentence and not like a stack trace.

Words used in this chapter

Container	It is a running copy of a program, kept apart from the rest of the machine, that brings its own operating-system files, libraries and settings with it, so it behaves the same way on any machine that can run Docker.
Image	It is the frozen, read-only package of the recipe plus all the parts that a container is started from — the image is the model on the shelf, and the container is the live copy running from it.
Multi-stage build	It is a Dockerfile written with two or more <code>FROM</code> sections, where the heavy tools (Maven, Node) live in an early stage and only the finished result is copied into the last stage, which keeps the shipped image small.
Reverse proxy	It is a server such as nginx that stands in front and passes some incoming requests on to another server behind it, so the outside world sees only one address.

Same origin / CORS	An origin is the protocol, the host and the port of a URL put together; CORS is the browser rule that stops a page from calling a different origin, and PMS never meets it because nginx serves the page and the API on the same origin.
Healthcheck	It is a small command that Docker runs inside a container again and again on a timer, to find out whether the program inside is really answering, so that other containers can wait for it.
CI/CD	Continuous Integration means a server builds and tests the project again by itself on every code change; Continuous Delivery means that same server builds the package you can deploy, by itself, as soon as the checks pass.
JaCoCo coverage	It is a Java tool that watches the tests run and tells you what share of the application's instructions, lines and branches really ran at least once — it measures how far the tests reach, not whether the code is right.

Traps to avoid

You might think the Docker image build runs the tests, but actually backend/Dockerfile says `mvn -B -ntp clean package -DskipTests` on purpose. Testing is the CI's job (the backend job runs `mvn verify`). The Dockerfile only packages. That is why a green image build on its own proves nothing about whether the code is right.

You might think the Angular app and the Spring Boot API are two different websites that need CORS setup, but actually nginx serves both of them under `http://localhost:8081` — the page comes from disk, and `/api/` is passed on to the backend container. The browser sees one single origin, so the cross-origin rules never start, and the refresh cookie with `SameSite=Strict` works with no special handling. The `CORS_ALLOWED_ORIGINS` variable in `docker-compose.yml` is only a left-over safety net for calls made straight to port 8091.

You might think the database healthcheck is boring boilerplate, but actually we had to force it onto TCP with `pg_isready -h 127.0.0.1`. Without that flag, the temporary server that PostgreSQL starts while it sets itself up answers the check. That temporary server only listens on a Unix socket. So the container is called healthy too early, and the backend starts against a database that is not taking clients yet. The `$$` on that line is not a typo either: it protects the variable so that the shell inside the container reads it, and not Docker Compose.

You might think 97 passing tests means the code is well covered, but actually JaCoCo reports only 31.9% instruction coverage, 41.2% line coverage and 24.5% branch coverage for the last run we recorded. About 11,000 of the instructions that are not covered are demo-data seeder classes. If you leave them out, the real coverage is 53.7% of instructions. Whole modules have no tests at all: `com.pms.mission` is at 0% and `com.pms.user.service` at 2.8%. There are also zero frontend tests — the workflow says so in a comment, because `angular.json` declares no test target. Be ready to say this openly, instead of letting a jury member find it.

You might think `docs/DEPLOYMENT.md` is the authority on the test count, but actually its CI table still says "compilation + 89 tests", while the code now has 97 `@Test` methods across 13 classes. Quote the code, not the doc.

One click, all the way down

Follow one real action through every layer you have just read about.

Everything in the seven chapters happens together, every time someone clicks a button. Below is one real action: a project manager adds a jalon (a billing step) worth 30% of the contract. You can follow it from the mouse all the way to the database and back. If you can tell this page again in your own words, you can explain the whole system.

A Chef de Projet opens the app and clicks "Facturation". In `frontend/pms-frontend/src/app/app.routes.ts` the entry path: 'billing' has data: { permission: 'VIEW_BILLING' } on it. `frontend/pms-frontend/src/app/core/guards/permission.guard.ts` reads that text and calls `auth.hasPermission(required)`. If he did not have it, he would end up on `/dashboard?forbidden=1`. The screen is `frontend/pms-frontend/src/app/features/billing/billing.component.ts`. He picks a project with `<app-project-picker>`. The "Add milestone" button shows up only because `canManage = () => this.auth.hasPermission('MANAGE_BILLING')` is true. That permission is given to CHEF_PROJET, and on purpose not to DIRECTEUR, by `backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql`.

He types "Acompte" and 30, and clicks save. `saveJalon()` does one small check on the screen (`if (!this.jalonForm.label || !this.jalonForm.pourcentage)`), then calls `createJalon` in `frontend/pms-frontend/src/app/core/services/billing.service.ts`, which POSTs to `/api/projects/7/jalons`. `frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts` sticks `Authorization: Bearer <token>` onto the request. `frontend/pms-frontend/docker/nginx.conf` sends `/api` on to `backend:8080`.

On the server, `backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java` builds his authorities again. Then

`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java`, registered in `WebMvcConfig.java`, matches `/api/projects/(\d+)` and calls `assertCanAccess(7, email)` in `backend/src/main/java/com/pms/project/service/ProjectScopeService.java`. That is a 403 if project 7 is not his. `backend/src/main/java/com/pms/billing/controller/BillingController.java` receives it. `@Valid` checks `backend/src/main/java/com/pms/billing/dto/JalonRequest.java` (`@NotBlank`, `@Positive`, `@DecimalMax("100")`).

`backend/src/main/java/com/pms/billing/service/JalonService.java` is where it is decided. First `@PreAuthorize("hasAuthority('MANAGE_BILLING'))`. Then `validatePourcentageSum` says no if the milestones of the project would go over 100%. Then `computeMontant` gives `budget × 30 / 100`.

The `JalonFacturation` that was built extends `BaseEntity`, so `created_by` is filled by `SpringSecurityAuditorAware`. It is saved through `JalonFacturationRepository` into the table made by `v9__schema_billing.sql`, whose `chk_jf_pourcentage` guards the value one last time. `JalonMapper` returns a `JalonResponse`, HTTP 201, and the component calls `reload()`.

Every word, in one place

All the words from the seven chapters, in one alphabetical list.

@PreAuthorize	A Java annotation put above a service method that makes Spring test a condition, such as <code>hasAuthority('VIEW_PROJECT')</code> , and refuse the call with HTTP 403 before the method body runs.
Access token	The short-lived JWT (15 minutes in PMS) that goes with every API call in the Authorization header and that carries the user's permission list.
Authority (Spring Security)	Spring's own name for one label stuck on the user who is logged in; in PMS each authority is exactly one permission code.
Authority / permission	A capability code such as <code>VIEW_PROJECT</code> or <code>MANAGE_DI</code> that is given to a role in the database and checked by the code; the code never checks a role name.
Avenant (contract amendment)	A paper signed with the client that changes the contract, and in the code it adds its amount to the project's revised budget and computes again every milestone that is still planned.
bcrypt hash	A slow-on-purpose one-way change of a password, kept in the <code>password_hash</code> column, from which you cannot get the first password back.
Budget effectif (effective budget)	The number the whole application computes with: the revised budget when there is at least one avenant, and the initial budget when there is none.
BusinessRuleException	The project's own error type, thrown when a company rule is broken, which cancels the whole database transaction and sends an error message back to the screen.
CA production	It is the revenue the company is allowed to treat as earned, equal to the value of the contract in TND multiplied by the EV percentage.
Capability (authority)	A named permission such as <code>MANAGE_BILLING</code> or <code>VALIDATE_WORKLOAD</code> that the code checks before it runs a method, instead of checking a job title like "director".
Charge (in JH, jour-homme / man-day)	It is an amount of work counted in days of one person's time, never a sum of money - 40 JH means someone worked forty days.
Charge réelle (actual workload)	The number of man-days a team member says he really spent on a project in one given month, which can no longer be changed once the chef de projet validates it.

Check constraint	A rule attached to a table, such as CHECK (pourcentage > 0 AND pourcentage <= 100), that PostgreSQL tests on every write and that makes the write fail if it is false.
CI/CD	Continuous Integration means a server builds and tests the project again by itself on every code change; Continuous Delivery means that same server builds the package you can deploy, by itself, as soon as the checks pass.
Component	One class that holds a screen's three parts together - the HTML template that is drawn, the TypeScript logic that answers clicks, and the CSS styles - which in PMS are all written inside one single .ts file under the @Component decorator.
Computed signal	A read-only signal whose value is worked out from other signals and worked out again by itself when any of them changes, such as sortedJalons() or filtered().
Container	It is a running copy of a program, kept apart from the rest of the machine, that brings its own operating-system files, libraries and settings with it, so it behaves the same way on any machine that can run Docker.
DTO (Data Transfer Object)	A plain object shaped for the outside world, like <code>ProjectResponse</code> , used so entities never leave the service layer and so the JSON sent to the browser can hide fields such as the budget.
EAC (Estimate At Completion)	It is today's best guess of what the project will have cost in total once it is finished, worked out as the cost already spent plus the cost of the work that is still planned.
Earned Value (EV %)	It is the share of the contract that has really been delivered, judged and typed in by the project manager at the monthly review instead of being computed by the machine.
Entity	A Java class such as <code>Project</code> whose fields match the columns of a database table one for one, so saving the object writes the row.
Flyway migration	A numbered SQL file (<code>v5__schema_project.sql</code>) that Flyway runs once and only once, in order, so any empty PostgreSQL instance can be rebuilt into exactly the schema you expect — 28 of them exist today.
Foreign key	A column that holds the primary key of a row in another table, so the database itself says no to a link that points at something that is not there.
Healthcheck	It is a small command that Docker runs inside a container again and again on a timer, to find out whether the program inside is really answering, so that other containers can wait for it.
HTTP interceptor	A function set up one time that every request and every answer goes through, used in PMS to add the Authorization header and to quietly get a new token when the old one has expired.

Image	It is the frozen, read-only package of the recipe plus all the parts that a container is started from — the image is the model on the shelf, and the container is the live copy running from it.
JaCoCo coverage	It is a Java tool that watches the tests run and tells you what share of the application's instructions, lines and branches really ran at least once — it measures how far the tests reach, not whether the code is right.
Jalon de facturation (billing milestone)	It is a slice of the contract, written as a percentage, that becomes an invoice amount of $\text{budget} \times \text{percentage} / 100$ and moves through the states PREVU, FACTURE and PAYE.
JH / jour-homme (man-day)	One person working one day, the unit used for planned workload, for real workload, and for backlog estimates.
JPA and Hibernate	JPA is the Java standard that says how a Java class can stand for a database table, and Hibernate is the library that does the real work of writing objects into rows and reading rows back into objects.
JWT (JSON Web Token)	A short signed piece of text the browser sends with every request to prove who the user is, so the server does not have to keep a session in memory for each connected user.
Layered architecture	A rule that the code inside one module is stacked in fixed layers — controller (HTTP), service (decisions), repository (database access), entity (table shape) — and a layer may only call the one right below it.
Lazy loading	Downloading a screen's JavaScript only the first time a user goes to it, written in PMS as <code>loadComponent: () => import('...')</code> inside the route.
Modular monolith	One single program you deploy, but cut inside into feature modules that stand on their own, chosen in ADR-016 instead of microservices because one team, one database and figures about money that always agree are easier to build, to test and to defend.
Multi-stage build	It is a Dockerfile written with two or more <code>FROM</code> sections, where the heavy tools (Maven, Node) live in an early stage and only the finished result is copied into the last stage, which keeps the shipped image small.
Observable / subscribe	An Observable is a plan for work that will give you values later; nothing is really sent to the server until someone calls <code>.subscribe()</code> , and the function you give to subscribe gets the answer.
Permission code	A short uppercase string such as <code>VIEW_ALL_PROJECTS</code> that lives as a row in the permissions table and is tied to roles through the <code>role_permissions</code> table.

Plan de charge (planned workload)	The plan of how many man-days each team member should spend on a project in one given month, typed in by the chef de projet (the project manager).
PPR (provision pour risques)	It is a risk cushion equal to 5% of the contract value in TND, shown on the project sheet as a reminder of how much room the project has before it is in trouble.
Primary key	The column, here always called id, whose value names one row and only that row, and which is never empty and never repeated.
RAF (reste a faire)	It is the number of man-days still to be worked, taken from the planned lines of the months that have not produced validated real days yet.
Refresh token	The long-lived JWT (7 days in PMS) kept in an HttpOnly cookie and used only to get a new access token, never to call a business endpoint.
Relational table	A grid kept inside the database where every column is one named piece of information and every row is one record, like one sheet in a register.
REST API	The set of web addresses the backend answers, where the address says what you want (<code>/api/projects/7</code>) and the HTTP verb says what to do with it (GET to read, POST to create, PUT to replace, PATCH to change one thing, DELETE to remove).
Reverse proxy	A server, here nginx, that sits in front and decides for each request coming in whether to answer it itself (the Angular files) or pass it to another server (<code>/api/...</code> goes to Spring Boot).
Route guard	A function the router runs before a screen shows up, which returns true to let it through or a redirect URL to send the user somewhere else - PMS uses <code>authGuard</code> (are you logged in), <code>permissionGuard</code> (do you have this permission) and <code>unsavedChangesGuard</code> (are you about to lose your edits).
Same origin / CORS	An origin is the protocol, the host and the port of a URL put together; CORS is the browser rule that stops a page from calling a different origin, and PMS never meets it because nginx serves the page and the API on the same origin.
Scope (périmètre)	The extra filter that decides which rows of data a user may see, on top of the question whether he may use the feature at all.
Signal	A box that holds a value and remembers which parts of the screen read it, so calling <code>.set()</code> or <code>.update()</code> on it redraws only those parts - and you read it by calling it like a function, for example <code>jalons()</code> .
Soft delete	Deleting by setting a true/false column named <code>deleted</code> to true instead of taking the row away, so the history stays easy to read and nothing that pointed at the row is broken.

Standalone component	A component that lists what it needs in its own imports: [...] line instead of being listed in one big shared NgModule file, which is why PMS has no *.module.ts files at all and every component says exactly what it uses.
Stateless	The server keeps nothing about your session between two requests, so it has to work out again who you are from the token you show each time (SessionCreationPolicy.STATELESS in SecurityConfig).
TCC (taux de charges complémentaires)	It is the extra percentage added on top of a person's raw daily rate to pay for social contributions, offices and management, so a rate of 0.20 means one day really costs 1.20 times the daily rate.
Tier	One of the three pieces of the system that run on their own — the browser application, the server application, and the database — where each one can be restarted, replaced or grown without rewriting the other two.
tokenVersion	A whole number kept both in the users table and inside every token; adding one to the column makes every token given to that user before stop working at once.